

智能体框架工程：基于可观测性驱动的编码智能体框架自动演化

Jiahang Lin^{1*†}, Shichun Liu^{1*†}, Chengjun Pan^{2*†}, Lizhi Lin³, Shihan Dou¹,
Zhiheng Xi¹, Xuanjing Huang¹, Hang Yan³, Zhenhua Han^{3†}, Tao Gui^{1†}, Yu-Gang Jiang¹

¹Fudan University ²Peking University ³Shanghai Qiji Zhifeng Co., Ltd

📧 china-qijizhifeng/agentic-harness-engineering

摘要

框架如今对智能体性能至关重要，它调节着模型与工具及执行环境的交互方式。然而，框架工程仍是一门手工技艺，因为自动化面临异构的动作空间（跨可编辑组件）、海量轨迹（掩盖了可操作信号）以及编辑效果难以归因等挑战。我们提出智能体框架工程（**Agentic Harness Engineering, AHE**），一个通过三大匹配的可观测性支柱应对这些挑战的闭环：①组件可观测性为每个可编辑框架组件提供文件级表示，使动作空间显式且可回退；②经验可观测性将数百万原始轨迹令牌蒸馏为分层、可下钻的证据语料库，供演化中的智能体实际消费；③决策可观测性为每次编辑配对一项自我声明的预测，并在下一轮任务级结果中验证。这些支柱共同将每次编辑转化为可证伪的契约，使框架演化自主进行，而不陷入试错。实验上，十轮 **AHE** 迭代将 **Terminal-Bench 2** 上的 **pass@1** 从 69.7% 提升至 77.0%，超越人工设计的框架 **Codex**（71.9%）以及自演化基线 **ACE** 和免训练 **GRPO**。冻结的框架无需重新演化即可迁移：在 **SWE-bench-verified** 上，它以比种子少 12% 的令牌用量实现最高聚合成功率；在 **Terminal-Bench 2** 上，它在三个替代模型家族中产生 +5.1 至 +10.1 个百分点的跨家族增益，表明演化出的组件编码了通用工程经验而非基准特定的调优。消融进一步将增益定位于工具、中间件和长期记忆，而非系统提示。这些结果将可观测性驱动的演化定位为一条实用路径，使编码智能体框架与其基础模型同步持续改进。

1 引言

编码智能体正越来越多地被部署于长周期软件工程任务中，在真实代码仓库的问题解决 [14, 46, 7] 和多步骤终端工作流 [21] 方面取得了可衡量的进展。在实践中，这种进展不仅依赖于底层语言模型，同样依赖于周围的工程组件：塑造工作风格的系统提示、暴露文件系统和外壳的工具，以及控制上下文、执行和恢复的中间件。这些模型外部、可编辑组件的集合统称为智能体的挂载装置 [29, 18, 42, 45, 33, 31]。

* 同等贡献。 † 通讯作者：hzhua201@gmail.com, tgui@fudan.edu.cn。 ‡ 工作于上海奇绩智峰有限公司实习期间完成。

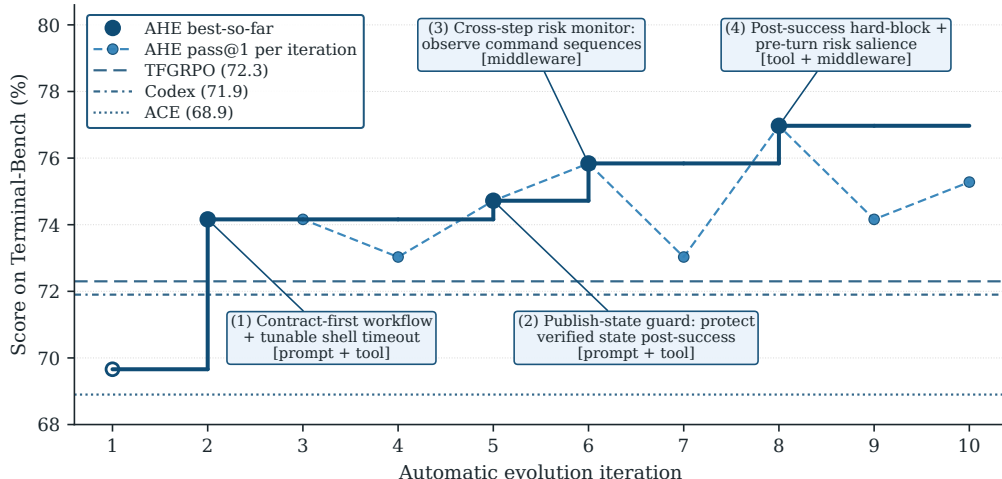


图 1: AHE 在 Terminal-Bench 2 上将一个仅使用 bash 的种子进化到超越所有人工设计和自我进化的基线。所有三个角色智能体共享一个基础模型，将增益隔离于挂载装置编辑，而非分析器或编辑器能力。

即使基础模型保持不变，挂载装置设计也会实质性地改变长周期编码基准上的任务完成情况 [40, 42]，这使得挂载装置工程成为改进编码智能体的一流杠杆。此外，最优挂载装置是模型特定的：为一个基础模型调优的挂载装置在另一个模型上往往表现不佳，并且必须随着基础模型的变化而重新适配。在当前实践中，这种适配是手动执行的——开发人员检查轨迹，识别反复出现的失败模式，并在提示、工具、中间件和技能之间手工制作编辑。然而，随着基础模型的快速进步 [39, 38, 44, 6, 35, 36]，这种手动循环难以跟上步伐，从而在模型能力与实现该能力所需的挂载装置之间造成了日益扩大的差距 [33]。

一个直观的方向是用一个进化智能体来自动化这个循环，该智能体基于经验优化挂载装置组件 [1, 49, 4]。然而，现有方法很少联合进化完整的可编辑组件集 [16]；大多数只关注单一组件，通常是提示 [32, 50, 20]、技能 [19, 43] 或上下文内剧本 [49]。端到端地联合进化多个组件面临两个结构性障碍：长而无结构的轨迹几乎不产生可操作的信号，而紧密耦合的挂载装置框架使得提示之外的编辑容易出错。这使得智能体驱动的挂载装置进化的核心问题悬而未决：进化智能体如何能够联合且稳定地进化编码智能体挂载装置的所有可编辑组件？

我们的核心洞见是，这一问题的瓶颈在于可观测性，而非智能体能力：一旦进化智能体在清晰的动作空间上获得结构化上下文，它就能可靠地收敛到更优的测试框架设计 [34, 53]。我们在智能体测试框架工程（AHE，图 2）中实现了这一点，这是一个由三大可观测性支柱驱动的闭环：①组件可观测性，通过解耦的测试框架将七种可编辑组件类型以文件形式暴露，使每种失败模式都能清晰地映射到单一组件类别；②经验可观测性，通过从数百万原始轨迹令牌中提炼出的分层、可下钻的证据语料库，使进化器消费结构化的根因而非原始日志；③决策可观测性，通过变更清单将每次编辑与自我声明的预测配对，并在下一轮任务级结果中验证，使每次编辑成为可证伪的契约，无效编辑在文件粒度上被回退。

我们在 Terminal-Bench 2 [21] 上对 AHE 进行了实证验证：十轮迭代将 pass@1 从 69.7% 提升至 77.0%，超越了人工设计的 Codex [25] 以及自进化基线 ACE [49] 和免训练 GRPO [4]。在未进一步进化的情况下，冻结的测试框架迁移至 SWE-bench 验证集 [14]，并在三个不同的基础模型家族上带来一致的 pass@1 增益，幅度为 +5.1 至 +10.1 个百分点，其中距离饱和更远的基础模型增益最大，这表明 AHE 编码了协调模式，而饱和度较低的模型更依赖这些模式。组件消融实验精确定位了增益来源：工具、中间件和长期记忆各自独立承载了改进，而仅靠系统提示则出现退化，说明事实性测试框架结构可跨任务和模型迁移，而散文式策略则不能。

本文做出三项贡献：

- 我们形式化了面向编码智能体的智能体驱动测试框架进化问题，并提出 AHE，该方法将组件、轨迹和决策三个层面的可观测性确定为设计核心，并通过三大可观测性支柱将每次测试框架编辑转化为可证伪的文件级契约：解耦的组件基座、分层轨迹蒸馏流水线，以及自我声明预测由下一轮任务增量验证的变更清单。
- 我们通过实验证明，AHE 将 Terminal-Bench 2 上的 pass@1 从 69.7% 提升至 77.0%，超越了人工设计和自动化的基线，并生成了一个可跨基准和基础模型系列迁移的冻结测试框架。
- 我们的分析揭示了智能体驱动进化的两个局限：测试框架组件之间的交互是非加性的，因此叠加有效的编辑会限制总体增益；循环的自我归因对于修复是可靠的，但对回归是盲目的，这指出了回归预见是未来自我进化循环最明确的方向。

2 相关工作

2.1 编码智能体的测试框架工程与评估

测试框架工程指的是设计围绕模型的系统的实践，包括其工具、接口、内存、执行约束和反馈循环，这些共同塑造了智能体在长时程任务上能够做什么 [29, 18, 40, 3, 33, 31]。具体而言，测试框架调节模型如何感知和作用于其环境：它暴露了工具增强推理展开所依据的动作和观察接口 [3]，用于仓库导航、文件编辑和命令执行的自定义智能体-计算机接口 [45]，以及保持长时程运行可复现的沙箱执行和编排支持 [42]。

验证此类系统确实有效推动了编码智能体评估沿两个轴的并行成熟：任务时程和环境真实性。覆盖范围从专注于污染和新鲜度控制的短时程函数级基准 [52, 12]，到仓库规模的可执行补丁解决 [14, 46, 7]，再到多小时的终端驱动工作流，这些工作流锻炼了长时程、真实的执行 [22, 5, 21]。并行的基础设施轨道围绕这些基准打包可执行运行时和验证器 [28, 13, 47]，其对可复现、可追踪和可验证执行的关注直接推动了 AHE 所构建的观察系统。

2.2 大语言模型智能体的自动优化

自动化智能体优化方法在优化器所观察的证据及其可编辑内容上存在差异。一些方法通过情景化批评与反思来修订智能体自身的输出 [20, 32, 9]。另一些方法则针对提示与指令 [15]：结构化操作手册 [49]、语义优势先验 [4]、面向多阶段程序的联合优化指令-演示流水线 [27]，以及由帕累托前沿迹驱动的反向更新 [1]。另一条研究路线直接编辑程序结构本身，其形式包括技能库 [41]、通过变异演化得到的评分程序与智能体档案 [24, 11]，以及从执行轨迹中搜索或学习得到的图结构工作流 [48, 51]。

AHE 将整个框架作为一个组合整体进行调优，而非单一可编辑表面，从而使跨组件权衡对优化器而言变得清晰可辨。同时，它保持人类先验的最小化，让方法论由优化器从执行轨迹中发现，而非人工固定。我们在第 3 节中描述了实现这些选择的基板、轨迹分析与迭代过程。

3 方法

AHE 将框架优化转化为由另一个智能体驱动的闭环过程，其中基础模型保持固定，仅编辑显式框架。我们的设计原则是，该闭环的每个阶段都必须是可观察的：AHE 忠实记录每个阶段产生的工件（某次迭代写入的框架组件、生成的执行轨迹、提交的编辑决策），并以结构化、分层的形式表示，使另一个智能体能够读取并据此行动。

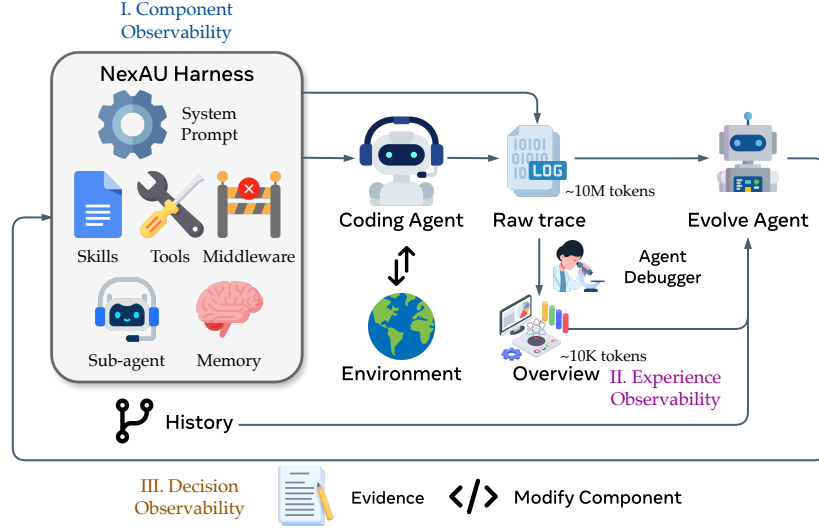


图 2：AHE 流水线将三个可观察表面连接为一个闭环。组件、执行经验与编辑决策均以结构化工件形式呈现，供另一个智能体读取，且每次编辑都成为下一轮可验证的可证伪预测。

三个可观察性层实现了这一原则。组件可观察性（§ 3.1）通过解耦的文件级框架基板实现，将每种失败模式映射到单一组件类。经验可观察性（§ 3.2）通过从原始执行轨迹中提炼的分层证据语料库实现，并建立索引以支持下钻访问。决策可观察性（§ 3.3）通过变更清单实现，将每次编辑与一个自声明的预测配对，由下一轮进行验证。这三层组合成算法 1 的迭代过程，可无人值守地逐轮运行。

3.1 NexAU：可编辑、解耦的框架基板

我们在 NexAU 框架 [23, 37] 上实例化了测试装置 H ，该框架在单一工作区中的固定挂载点以显式文件形式暴露了七种正交组件类型：系统提示、工具描述、工具实现、中间件、技能、子代理配置和长期记忆。这些组件类型之间松散耦合，因此添加中间件无需编辑系统提示，添加技能也无需改动任何工具。

这种解耦实现了组件可观测性：每种失败模式都映射到单一组件类，为进化代理提供了清晰的动作空间，并将每次通过率变化定位到单个文件，而不是分散在数百行非结构化的提示文本中。每次逻辑编辑都成为工作区 git 历史中的一次提交，从而免费获得文件级差异和回滚粒度。

我们的种子测试装置 H_0 刻意保持极简：仅包含一个 shell 执行工具，没有中间件、技能或子代理。一个已经适配目标基准的种子会污染后续每次编辑的归因，因为我们无法判断收益是来自循环还是来自种子。极简种子迫使 AHE 添加的每个组件都必须通过实测回放来证明其价值。

3.2 代理调试器：分层轨迹证据

我们使用测试装置 H 为基准中的每个任务生成 k 轨迹，这些轨迹可能包含由测试装置缺陷导致的错误，这些错误可以采取行动，但分散在数百万个原始消息令牌中。为了从代理轨迹中提取洞见并实现经验可观测性，我们应用代理调试器 [17] 框架，使用一个代理来探索被构建为可导航的基于文件环境的轨迹，其中每条轨迹消息都位于自己的文件中，并通过通用 shell 和脚本工具访问。具有相同查询的轨迹被放置在同一环境中，调试器需要分析失败或成功模式的根本原因，并将其存储在每任务的分析报告中。分析还包括通过/失败

任务的状态，为进化代理提供依据。最后，从每份报告中汇总出基准级别的概览，形成单一文档，作为每次迭代的入口点。

除了这些报告，我们还提供原始轨迹，以便智能体需要验证报告中的声明时使用。轨迹以原始形式和经过轻度处理的形式提供，以去除不必要的内容。所有这些内容都以文件形式提供，允许渐进式披露 [30]，从而节省令牌并支持更好的智能体决策。

3.3 进化智能体：证据驱动、可审计的编辑

进化智能体闭合了自适应框架增强循环。在每一轮中，它读取由智能体调试器生成的分层证据语料库，决定添加、修改或删除哪些框架组件，将这些编辑应用到工作区，并记录每次编辑背后的推理。两个约束支配这些编辑，它们共同实现了决策可观测性：每次编辑都成为记录在版本化清单中的可证伪的文件级声明，下一轮的裁决要么确认它，要么回滚它。

第一个约束是可控性：进化智能体只在框架工作区内写入，而运行目录、追踪器、验证器和语言模型配置是只读的，种子系统提示（附录 B.1）被标记为不可删除。这些限制阻止了不受约束的自我修改者会采取的捷径，例如禁用验证器、更换模型或提高推理预算，并确保每个记录的增益都归因于框架编辑。

第二个约束是每次更改都是证据驱动的，并附带记录的预测。每次编辑附加一个清单条目，其中指明失败证据、推断的根本原因、目标修复以及包含预期修复和风险回归的预测影响；该清单是循环的证据账本（见附录 B.2）。在下一轮中，循环将预测修复集和预测回归集与观察到的任务级差异相交，以产生每次编辑的裁决。因此，每次编辑都可通过下一次评估进行证伪，这用轮次之间的可衡量契约取代了基于理由的自我辩护。

Algorithm 1 AHE outer loop.

Require: seed harness H_0 , base model M , benchmark D , rollouts per task k , max iterations N

```

1:  $H_{\text{best}} \leftarrow H_0$ 
2: for  $t = 1$  to  $N$  do
3:    $T_t \leftarrow \text{ROLLOUT}(M, H_{t-1}, D, k)$  ▷ phase 1:  $k$  rollouts per task
4:    $\tilde{T}_t \leftarrow \text{CLEAN}(T_t)$  ▷ phase 2: normalize traces into canonical form
5:   if  $t \geq 2$  then ▷ phase 3: attribute prior manifest, then rollback
6:      $V_t \leftarrow \text{ATTRIBUTE}(C_{t-1}, T_{t-1}, T_t)$ 
7:      $H_{t-1} \leftarrow \text{ROLLBACK}(H_{t-1}, V_t)$ 
8:   else
9:      $V_t \leftarrow \emptyset$ 
10:  end if
11:   $R_t \leftarrow \text{AGENTDEBUGGER}(\tilde{T}_t)$  ▷ phase 4: layered distillation
12:   $(H_t, C_t) \leftarrow \text{EVOLVE}(H_{t-1}, R_t, V_t)$  ▷ phase 5: workspace edits + new manifest
13:   $\text{COMMIT}(H_t, C_t, t)$  ▷ phase 6: tag iteration in git
14:  if  $\text{PASS@1}(T_t) > \text{PASS@1}(H_{\text{best}})$  then  $H_{\text{best}} \leftarrow H_t$ 
15:  end if
16: end for
17: return  $H_{\text{best}}$ 

```

Algorithm 1 composes the three substrates into one iteration: rollout, clean, attribute the prior manifest and revert rejected edits, distill, edit, commit. We run $k \geq 2$ rollouts per task so each task carries a pass-rate signal, which stabilizes pass@1 and lets partial-pass tasks anchor comparative diagnosis. Attribution runs *before* distillation, so its verdict lands inside the evidence corpus and binds each prior manifest entry as a contract rather than a rationale. A one-shot explore agent (Appendix B.3) runs in parallel with iteration 1 to seed a small number of reusable skills from the NexAU source and public coding-agent references. These skills receive no special protection: from iteration 2 onward the Evolve Agent may keep, refine, or remove them based on observed rollouts.

4 实验

我们围绕三个问题组织实证研究：自适应框架增强在现有框架设计方法图谱中的位置，其产物是否可移植到优化目标之外，以及循环内部什么驱动了增益。

Research Questions

1. 研究问题一（第 4.2 节）：为何采用智能体式测试框架工程，而非人工设计的测试框架或其他自动化方法？2. 研究问题二（第 4.3 节）：智能体式测试框架工程是否会对其优化目标产生过拟合？3. 研究问题三（第 4.4 节）：智能体式测试框架工程内部哪些因素驱动了性能提升，且该循环的自我归因可靠性如何？

4.1 实验设置

评估。我们在终端基准 2 (Terminal-Bench 2) [21] 的全部 89 个任务上驱动演化，任务难度划分为 4 个简单、55 个中等和 30 个困难，每个任务的超时时间延长至 1 小时。为进行跨基准迁移评估，我们在软件工程基准验证集 (SWE-bench-verified) [14] 上评估智能体式测试框架工程生成的测试框架，该验证集包含七个代码仓库的 500 个任务。我们为每种配置报告两项度量：通过率@1 (pass@1)，即每个任务 k 次尝试的平均二元成功率；以及令牌数/试验 (tokens/trial)，即所有大语言模型调用中提示与补全令牌总数的平均值，以千为单位。基础设施中止或超时的试验在通过率@1 下计为失败（与官方终端基准排行榜一致），并从令牌均值中排除，以避免截断数据。运行时基础设施（框架、调度器、沙箱、追踪器及并发）详见附录 A。

模型。在演化循环和第 4.2 节的主实验中，三种角色智能体（代码智能体、智能体调试器和演化智能体）共享同一个基础模型，即高推理设置下的 GPT-5.4 [26]。对于跨模型迁移（第 4.3 节），我们在五个替代基础模型上重新评估代码智能体：中等和超高推理设置下的 GPT-5.4 [26]、qwen-3.6-plus [38, 44]、gemini-3.1flash-lite-preview [8] 以及 deepseek-v4-flash [6]。

4.2 研究问题一：主要结果

我们从仅含 bash 的 **NexAU0** 种子 (§ 3.1) 出发，在 Terminal-Bench 2 上运行了一次包含十次迭代的 AHE 活动，大约 32 小时完成；报告的最佳配置记为 AHE。两个自进化基线 ACE [49] 和无训练 GRPO (TF-GRPO) [4] 从相同的 **NexAU0** 种子开始。

AHE outperforms both human-designed and self-evolved harnesses.

AHE 在我们的评测面板上优于所有基线：三个人工设计的测试框架 OpenCode [2]、Terminus-2 [10] 和 Codex [25]，以及两个自进化基线。图 1 显示增益随迭代累积，持续进化使 pass@1 进一步高于 NexAU0 种子。按难度划分，唯一的例外是困难层级，AHE 略微落后于 Codex。我们将这一差距

归因于 AHE 组件在长程任务上的相互干扰，而非能力缺失：仅将 AHE 的长期记忆单独替换到 NexAU0 种子中，而不加入其他 AHE 组件，就已经在困难层级上超过 Codex (§ 4.4.1)。

仅提示词的自进化遗漏了承载智能体框架工程 (AHE) 增益的组件。与 ACE 和 TF-GRPO 的差距源于层级不匹配。ACE 蒸馏出智能体阅读的自然语言操作手册

表 1: 终端基准 2 上 89 个任务的通过率@1，按官方难度划分。NexAU0 为共享种子；ACE、无训练 GRPO 和 AHE 是在其之上叠加的三种自演化循环。每列最佳结果以粗体标出；并列最佳均以粗体显示。

Method	All 89	Easy 4	Med. 55	Hard 30
Human-designed harness				
OpenCode	47.2%	75.0%	52.7%	33.3%
Terminus-2	62.9%	75.0%	74.5%	40.0%
Codex	71.9%	75.0%	80.0%	56.7%
Self-evolved from NexAU0				
NexAU0	69.7%	87.5%	78.2%	51.7%
ACE	68.9%	91.7%	78.2%	48.9%
TF-GRPO	72.3%	100.0%	79.4%	55.6%
AHE	77.0%	100.0%	88.2%	53.3%

表 2：在 SWE-bench-verified 上的跨基准迁移。ACE、无训练 GRPO (TF-GRPO) 和 AHE 共享 NexAU0 种子，仅在自进化循环上有所不同；四列均在 GPT-5.4 上运行。AHE 和两个自进化基线在 Terminal-Bench 2 上进化，并在无领域内再进化的情况下评估。每列加粗标记最佳；并列时均加粗。

Repo	N	Success rate $\uparrow$				Tokens k $\downarrow$			
		ACE	TF-GRPO	NexAU ₀	AHE	ACE	TF-GRPO	NexAU ₀	AHE
All	500	74.6%	74.2%	75.2%	75.6%	679	582	526	461
django	231	79.2%	78.8%	79.2%	81.0%	707	583	527	484
sympy	75	69.3%	68.0%	70.7%	70.7%	602	572	494	479
sphinx-doc	44	61.4%	65.9%	68.2%	70.5%	990	848	731	656
matplotlib	34	70.6%	70.6%	73.5%	73.5%	622	530	486	391
scikit-learn	32	93.8%	93.8%	93.8%	87.5%	451	378	307	257
pydata	22	77.3%	77.3%	77.3%	72.7%	563	516	386	338
astropy	22	59.1%	59.1%	54.5%	50.0%	546	470	667	277

在上下文中，而 TF-GRPO 是 GRPO 的一种轨迹反馈变体，强化成功的工具序列；这两种方法都没有开放周围脚手架进行编辑。AHE 则联合进化系统提示、工具、中间件和长期记忆，§ 4.4.1 显示增益集中在后三个组件，正是 ACE 和 TF-GRPO 未触及的层级。

4.3 研究问题 2：迁移到未见任务和基础模型

AHE 的框架是在 Terminal-Bench 2 上使用 GPT-5.4 高等级演化而来的。我们通过将演化后的框架原封不动地迁移到两个非目标设置中，来探究它编码的是通用的编码智能体经验，还是对目标任务的过拟合：一个不同的任务表面（SWE-bench-verified）和三个替代的基础模型。

跨基准迁移。我们在 SWE-bench-verified 上，在相同的基础设施下，将 AHE 框架与种子以及两个自演化基线（NexAU0、ACE、TF-GRPO）进行评估（表 2）。

ACE 和 TF-GRPO 在总体成功率上均退化到 NexAU0 种子以下，同时比种子多消耗 11% 到 29% 的令牌：ACE 注入的剧本和 TF-GRPO 强化的轨迹分布是在 Terminal-Bench 2 的迹上蒸馏得到的，并且在每次模型调用时都附着在提示上，因此在不同的任务表面上，这些文本增加了成本，却没有重塑底层策略。

AHE 反而取得了最高的总体成绩，相对于种子的增益集中在 django 和 sphinx-doc 上，这是两个最大且令牌开销最高的仓库，其多步骤编辑-验证循环与 AHE 在 Terminal-Bench 2 上压缩的工具、中间件和长期记忆的结构相匹配。仅在三个最小的仓库上出现了边际退化，这与小仓库上 pass@1 方差超过每个仓库增益的情况一致。AHE 还将总体令牌消耗相对于 ACE 削减了 32%，相对于 TF-GRPO 削减了 21%，相对于种子削减了 12%：将行为编码在工具、中间件和记忆中，避免了仅基于提示的基线所承担的每次调用重新推导的成本。

跨模型迁移。我们在 § 4.1 中列出的五个替代基础上重新评估了 NexAU0 种子和 AHE。图 3 报告了从 +2.3 到 +10.1 pp. 的五个正 pass@1 增益。

跨家族增益超过了家族内增益：deepseek-v4-flash 从 51.7% 移动到 61.8%，提升了 +10.1 个百分点；qwen-3.6-plus 从 56.2% 移动到 62.5%，提升了 +6.3 个百分点；gemini-3.1-flash-lite-preview 从 36.5% 移动到 41.6%，提升了 +5.1 pp 个百分点，均高于 GPT-5.4 medium 和 xhigh 上的 +2.3 个百分点。我们将此解读为：距离饱和更远的基础模型更多地依赖 AHE 在工具、中间件和长期记忆中固定的协调模式，而更强的基础模型则以较低的边际成本从其提示中重新推导出相同的协调。

在同一家族内，性能曲线呈非单调变化：中等规模上为 +2.3 个百分点，大规模上为 +7.3 个百分点（见第 4.2 节），超大规模上为 +2.3 个百分点。AHE 的步骤预算和单任务超时在进化过程中针对 GPT-5.4 大规模进行了适配；中等规模拥有更多的每步时间余量，但损失了一个原始能力推理层级，而超大规模则使更多试验超出单任务超时，根据第 4.1 节的 pass@1 约定，这些均计为失败。任一方向的调整都会削弱增益。

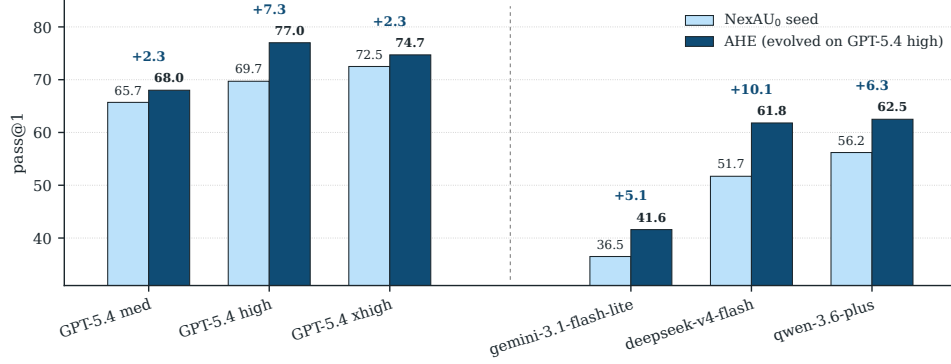


图 3：在 Terminal-Bench 2（89 个任务）上的跨模型迁移。在 GPT-5.4 高规模上进化得到的 AHE 工作区，在不进行进一步进化的情况下，直接在每个基座模型上重新评估，并与同一基座上的 NexAU₀ 种子进行对比。所有基座模型均较种子有所提升，且跨家族基座模型的提升幅度大于家族内基座模型。

核心发现是，所有五项增益均为正值：AHE 工作区并非特定于某一提供商的惯用模式或某一推理深度。其增益幅度与进化操作点相关，而非与基座模型的原始能力相关，因此我们将超时预算耦合视为泛化风险，并在局限性章节中讨论。

4.4 研究问题 3：分析

我们围绕第 3 节所强调的两个架构选择对循环进行分析：分解组件（第 4.4.1 节）和自声明归因（第 4.4.2 节）。

4.4.1 研究问题 3a：各组件中价值的累积位置

表 3 通过隔离四个进化层（长期记忆、工具、中间件或系统提示）之一，同时保持其余三个为种子默认值，在组件层面分解了 AHE 的增益。四个单组件变体中有三个优于种子；系统提示替换是唯一出现性能回退的情况。

表 3：在 Terminal-Bench 2 上的组件级消融实验。每个“仅+ X”行表示将单个 AHE 组件替换到 NexAU₀ 种子中；每列最佳结果以粗体显示。大多数单组件替换已优于种子，且单组件增益超过了完整 AHE 的总体增益，表明组件之间并非简单叠加，而是存在非加性交互。

Variant	All 89 tasks	Easy 4 tasks	Medium 55 tasks	Hard 30 tasks
NexAU ₀	69.7%	87.5%	78.2%	51.7%
+ memory only	75.3%	50.0%	83.6%	63.3%
+ tool only	73.0%	75.0%	87.3%	46.7%
+ middleware only	71.9%	100.0%	81.8%	50.0%
+ system_prompt only	67.4%	75.0%	78.2%	46.7%
AHE full	77.0%	100.0%	88.2%	53.3%

每个组件拥有不同的失效面。记忆模块增加了 12 条边界案例经验（性能裕度、排队超限取消、评估器式闭包、源打包布局）；在困难任务上这些经验使其超越完整 AHE，而在简单任务上则沦为多余的重叠验证。工具模块成为一个 1364 行的外壳，自动从每个命令附近的文件中浮现契约提示；在中等任务上它落在完整 AHE 的 0.9 个百分点以内，而在困难任务上内置的发布守卫过早地闭合了循环。中间件模块增加了一个完成钩子，强制进行一次评估器同构的闭包检查；在简单任务上它清除了所有任务，而在困难任务上它增加了回合数。系统提示编码了 79 行通用纪律，其可执行性取决于其他三个模块；单独插入时它获得 -2.3 个百分点的聚合分数。

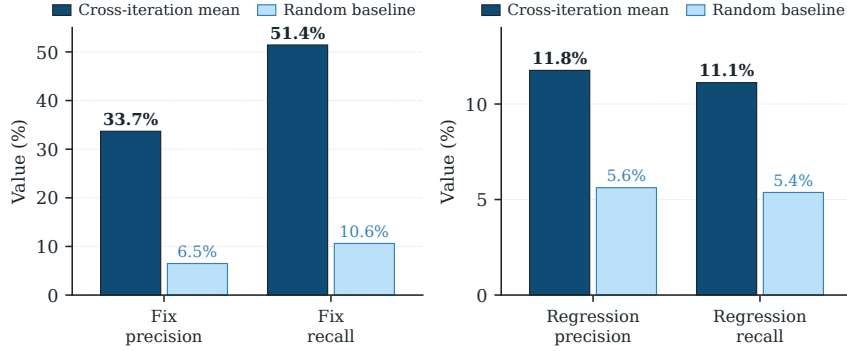


图 4：在 GPT-5.4 AHE 循环于 Terminal-Bench 2 上的 9 轮评估中，进化模型自我预测的跨迭代平均精确率和召回率，以及随机预测基线。左：修复预测。右：回归预测。

组件以非加性方式交互，限制了聚合增益。三个正向的单组件增益之和为 +11.1 个百分点，而完整 AHE 为 +7.3 个百分点；在困难任务上，仅记忆模块的变体超过了完整 AHE：记忆模块、中间件和系统提示都朝着相同的闭包式验证推进，因此堆叠它们会在长时程预算内花费回合进行冗余的重复检查。由于进化智能体优化的是由 55 个中等任务主导的聚合指标，它收敛到一个偏向中等任务的权衡，从而回吐了部分困难任务上的记忆效应，我们将交互感知的进化留给未来工作。

4.4.2 研究问题 3b：循环的自我归因追踪现实的可靠性

每一轮进化中，我们的进化模型生成一份变更清单，指明它预期在下一轮修复哪些 Terminal-Bench 2 任务，以及标记哪些任务存在回归风险。我们将第 $N-1$ 轮的预测与第 N 轮的真实情况进行比较，在 89 个任务上分别计算修复和回归的标准精确率和召回率。

证据驱动的目标定位。图 4 的修复面板显示，演化模型的目标定位是基于证据而非猜测。跨迭代修复精确率为 **33.7%**，修复召回率为 **51.4%**，大约是随机预测基线 6.5% 和 10.6% 的 5 倍，因此每次测试框架编辑都落在真实且智能体预期的目标上，而非面板的任意子集。

回归盲区。回归面板呈现相反的情况：跨迭代回归精确率为 **11.8%**，回归召回率为 **11.1%**，仅为随机基线 5.6% 和 5.4% 的 2 倍，因此大多数即将发生的回归未被预见。智能体能够解释为何某项编辑应有助于改进，但无法可靠指出同一编辑即将破坏的任务，这正是 § 4.2 演化曲线中出现非单调步骤的原因。缩小这一差距是未来自演化循环最明确的方向。附录 D 给出了逐轮分解。

5 结论

本文提出了智能体测试框架工程（Agentic Harness Engineering, AHE），这是一个以可观测性驱动的循环，在基础模型保持不变的情况下，将编码智能体的测试框架转化为可学习的适应表面。AHE 将组件以文件形式暴露，将执行轨迹提炼为分层证据语料库，并将每次编辑与可证伪的下一轮预测绑定；十次迭代将 Terminal-Bench 2 上的 pass@1 从 69.7% 提升至 77.0%，且冻结的测试框架可迁移至 SWE-bench-verified 及三个替代模型家族。我们认为测试框架层面的演化是模型侧训练的补充维度：一个外部化、可审计的表面，编码智能体的经验可在此积累。

局限性

本研究探索了一个有前景但高方差的场景，因此对结论范围的解读应相应谨慎。

基准范围。我们的评估推动了终端基准 2 (Terminal-Bench 2) 的演进, 并探测了在软件工程基准验证集 (SWE-bench-verified) 上的迁移能力。尽管冻结的测试框架能够迁移到第二个任务表面和三个替代的基础模型家族, 但更广泛的编程语言、仓库规模的部署以及人在回路 (human-in-the-loop) 工作流仍未得到测试。

演进操作点。自适应超参数演进 (AHE) 的步骤预算和每任务超时在演进期间针对 GPT-5.4 高配版进行了拟合, 因此跨模型迁移数据混淆了框架可移植性与操作点耦合——在同一家族内, 增益在推理层级上呈现非单调性 (§ 4.3)。要解开这些因素, 需要在多个操作点下重新运行循环。

自我修改治理。自适应超参数演进 (AHE) 将编辑限制在工作区内, 在版本化清单中记录每次更改, 并在文件粒度上回滚无效编辑, 但它并未提供完整的护栏机制栈。长期框架清理和更强的滥用预防仍不完整, 应将 AHE 视为受控的研究原型, 而非完全成熟的自主自我改进系统。

参考文献

- [1] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, October 2025. URL <https://openreview.net/forum?id=RQm2KQTM5r>.
- [2] Anomaly. Opencode: The open source coding agent., 2025. URL <https://github.com/anomalyco/opencode>.
- [3] Anthropic. Claude-code, 2025. URL <https://github.com/anthropics/claude-code>.
- [4] Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, Yong Mao, Ke Li, and Xing Sun. Training-free group relative policy optimization, October 2025. URL <http://arxiv.org/abs/2510.08191>.
- [5] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=6s5uXNWGIh>.
- [6] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, April 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf.
- [7] Xiang Deng, Jeff Da, Edwin Pan, Yannis Y. He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa R. Kundurthy, Sean M. Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? October 2025. URL <https://openreview.net/forum?id=9R2iUHhVfr>.
- [8] Google. Gemini-3-1-flash-lite-model-card, March 2026. URL <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-1-Flash-Lite-Model-Card.pdf>.
- [9] Honglin Guo, Kai Lv, Qipeng Guo, Tianyi Liang, Zhiheng Xi, Demin Song, Qiuyinzhe Zhang, Yu Sun, Kai Chen, Xipeng Qiu, and Tao Gui. Critiq: Mining data quality criteria from human preferences. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16240–16261, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.792. URL <https://aclanthology.org/2025.acl-long.792/>.

- [10] Harbor. Terminus-2, 2026. URL <https://www.harborframework.com/docs/agents/terminus-2>.
- [11] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=t9U3LW7JVX>.
- [12] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- [13] Naman Jain, Jaskirat Singh, Manish Shetty, Tianjun Zhang, Liang Zheng, Koushik Sen, and Ion Stoica. R2e-gym: Procedural environment generation and hybrid verifiers for scaling open-weights swe agents. In *Second Conference on Language Modeling*, August 2025. URL <https://openreview.net/forum?id=7evvwdo3z#discussion>.
- [14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, October 2023. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [15] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, October 2023. URL <http://arxiv.org/abs/2310.03714>.
- [16] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, March 2026. URL <http://arxiv.org/abs/2603.28052>.
- [17] Lizhi Lin. Agent debugger: Understanding agent trajectory with agentic workflows - dawning road, February 2026. URL <https://dawning-road.github.io/blog/agent-debugger>.
- [18] Ryan Lopopolo. Harness engineering: Leveraging codex in an agent-first world, February 2026. URL <https://openai.com/zh-Hans-CN/index/harness-engineering/>.
- [19] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver, April 2026. URL <http://arxiv.org/abs/2604.08377>.
- [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL <https://openreview.net/forum?id=S37h0erQLB>.
- [21] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis, Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu,

- Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, January 2026. URL <http://arxiv.org/abs/2601.11868>.
- [22] Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. Swe-lancer: Can frontier llms earn \$1 million from real-world freelance software engineering? In *Forty-Second International Conference on Machine Learning*, June 2025. URL <https://openreview.net/forum?id=xZXhFg43EI>.
 - [23] Nex-AGI. Nexau (au for agent universe), a general-purpose agent framework for building intelligent agents with tool capabilities., 2025. URL <https://github.com/nex-agi/NexAU>.
 - [24] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, June 2025. URL <http://arxiv.org/abs/2506.13131>.
 - [25] OpenAI. Codex cli, 2025. URL <https://developers.openai.com/codex/cli>.
 - [26] OpenAI. Introducing gpt-5.4, March 2026. URL <https://openai.com/index/introducing-gpt-5-4/>.
 - [27] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.525. URL <https://aclanthology.org/2024.emnlp-main.525/>.
 - [28] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. In *Forty-Second International Conference on Machine Learning*, June 2025. URL <https://openreview.net/forum?id=Cq1BNvHx74>.
 - [29] Prithvi Rajasekaran. Harness design for long-running application development, March 2026. URL <https://www.anthropic.com/engineering/harness-design-long-running-apps>.
 - [30] Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, Jeremy Hadfield, Rafi Ayub, Hannah Moran, Cal Rueb, Connor Jennings, Molly Vorwerck, Stuart Ritchie, and Maggie Vo. Effective context engineering for ai agents, September 2025. URL <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
 - [31] Nous Research. Hermes agent — the agent that grows with you, 2026. URL <https://hermes-agent.nousresearch.com/>.
 - [32] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL <https://openreview.net/forum?id=vAElhFcKW6>.
 - [33] Peter Steinberger. Openclaw — personal ai assistant, February 2026. URL <https://openclaw.ai/>.
 - [34] Rich Sutton. The bitter lesson, March 2019. URL https://www.cs.utexas.edu/~eunsol/courses/data/bitter_lesson.pdf.
 - [35] Kimi Team. Kimi k2.6 tech blog: Advancing open-source coding, April 2026. URL <https://www.kimi.com/blog/kimi-k2-6>.

- [36] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, Yanru Chen, Yanxu Chen, Yicun Chen, Yimin Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Ziwei Chen, Dazhi Cheng, Minghan Chu, Jialei Cui, Jiaqi Deng, Muxi Diao, Hao Ding, Mengfan Dong, Mengnan Dong, Yuxin Dong, Yuhao Dong, Angang Du, Chenzhuang Du, Dikang Du, Lingxiao Du, Yulun Du, Yu Fan, Shengjun Fang, Qiulin Feng, Yichen Feng, Garimugai Fu, Kelin Fu, Hongcheng Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Chengyang Gong, Xiaochen Gong, Zhuoma Gongque, Qizheng Gu, Xinran Gu, Yicheng Gu, Longyu Guan, Yuanying Guo, Xiaoru Hao, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Jiaxi Hu, Yangyang Hu, Zhenxing Hu, Ke Huang, Ruiyuan Huang, Weixiao Huang, Zhiqi Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yu Jing, Guokun Lai, Aidi Li, C. Li, Cheng Li, Fang Li, Guanghe Li, Guanyu Li, Haitao Li, Haoyang Li, Jia Li, Jingwei Li, Junxiong Li, Lincan Li, Mo Li, Weihong Li, Wentao Li, Xinhao Li, Xinhao Li, Yang Li, Yanhao Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zheming Li, Weilong Liao, Jiawei Lin, Xiaohan Lin, Zhishan Lin, Zichao Lin, Cheng Liu, Chenyu Liu, Hongzhang Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Tianyu Liu, Weizhou Liu, Xiangyan Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yuanxin Liu, Yue Liu, Zhengying Liu, Zhongnuo Liu, Enzhe Lu, Haoyu Lu, Zhiyuan Lu, Junyu Luo, Tongxu Luo, Yashuo Luo, Long Ma, Yingwei Ma, Shaoguang Mao, Yuan Mei, Xin Men, Fanqing Meng, Zhiyong Meng, Yibo Miao, Mingqing Ni, Kun Ouyang, Siyuan Pan, Bo Pang, Yuchao Qian, Ruoyu Qin, Zeyu Qin, Jiezhong Qiu, Bowen Qu, Zeyu Shang, Youbo Shao, Tianxiao Shen, Zhennan Shen, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Feifan Song, Pengwei Song, Tianhui Song, Xiaoxi Song, Hongjin Su, Jianlin Su, Zhaochen Su, Lin Sui, Jinsong Sun, Junyao Sun, Tongyu Sun, Flood Sung, Yunpeng Tai, Chuning Tang, Heyi Tang, Xiaojuan Tang, Zhengyang Tang, Jiawen Tao, Shiyuan Teng, Chaoran Tian, Pengfei Tian, Ao Wang, Bowen Wang, Chensi Wang, Chuang Wang, Congcong Wang, Dingkun Wang, Dinglu Wang, Dongliang Wang, Feng Wang, Hailong Wang, Haiming Wang, Hengzhi Wang, Huaqing Wang, Hui Wang, Jiahao Wang, Jinhong Wang, Jiuzheng Wang, Kaixin Wang, Linian Wang, Qibin Wang, Shengjie Wang, Shuyi Wang, Si Wang, Wei Wang, Xiaochen Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yipu Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhexu Wang, Zihan Wang, Zizhe Wang, Chu Wei, Ming Wei, Chuan Wen, Zichen Wen, Chengjie Wu, Haoning Wu, Junyan Wu, Rucong Wu, Wenhao Wu, Yuefeng Wu, Yuhao Wu, Yuxin Wu, Zijian Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Yuchong Xie, Yifei Xin, Bowei Xing, Boyu Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinbo Xu, Xinran Xu, Yangchuan Xu, Yichang Xu, Yuemeng Xu, Zelai Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Guangyao Yang, Hao Yang, Junwei Yang, Kai Yang, Ningyuan Yang, Ruihan Yang, Xiaofei Yang, Xinlong Yang, Ying Yang, Yi Yang, Yi Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Dan Ye, Wenjie Ye, Zhuorui Ye, Bohong Yin, Chengzhen Yu, Longhui Yu, Tao Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Xiaokun Yuan, Yang Yue, Weihao Zeng, Dunyuan Zha, Haobing Zhan, Dehao Zhang, Hao Zhang, Jin Zhang, Puqi Zhang, Qiao Zhang, Rui Zhang, Xiaobin Zhang, Y. Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yushun Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Chenguang Zhao, Feifan Zhao, Jinxiang Zhao, Shuai Zhao, Xiangyu Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Junfeng Zhong, Longguang Zhong, Weiming Zhong, M. Zhou, Runjie Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yuxuan Zhu, Zhen Zhu, Jingze Zhuang, Weiyu Zhuang, Ying Zou, and Xinxing Zu. Kimi k2.5: Visual agentic intelligence, February 2026. URL <http://arxiv.org/abs/2602.02276>.
- [37] Nex-AGI Team, Yuxuan Cai, Lu Chen, Qiaoling Chen, Yuyang Ding, Liwen Fan, Wenjie Fu, Yufei Gao, Honglin Guo, Pinxue Guo, Zhenhua Han, Zhengfu He, Hanglei Hu, Kai Hu, Shengjia Hua, Tianyu Huai, Baodai Huang, Li Ji, Zhen Jiang, Zhikai Lei, Bufan Li, Jiahang Lin, Lizhi Lin, Jinxiu Liu, Shichun Liu, Ziming Liu, Yuchen Ni, Pengfang Qian, Yujiong Shen, Qingyun Shi, Wentao Shu, Peng Sun, Yiran Suo, Tian Tang, Boyu Tian, Guoteng Wang, Junzhe Wang, Peixin Wang, Zhiheng Xi, Hang Yan, Jie Yang, Zhixiong Yang, Tianchu Yao, Guangze Ye, Qianxi Yu, Shuo Zhang, Xinyue Zhang, Yiqi Zhang, Jiarong Zhao, Miao Zheng, Rui Zheng, Enyu Zhou, Jiazheng Zhou, Maosen Zhou, Yuhao Zhou, Tao Gui, Yining Zheng, Xinchu Chen, Jie Zhou, Siyuan Feng, Qin Chen, Liang He, Qi Zhang, Xuanjing Huang, and Xipeng Qiu. Nex-1: Agentic models trained via a unified ecosystem for large-scale environment construction,

- December 2025. URL <http://arxiv.org/abs/2512.04987>.
- [38] Qwen Team. Qwen3.6-plus: Towards real world agents, April 2026. URL <https://qwenlm.github.io/blog/qwen3.6/>.
 - [39] Xiaomi MiMo Team. MIMO-v2.5-pro, April 2026. URL <https://huggingface.co/XiaomiMiMo/MiMo-V2.5-Pro>.
 - [40] Vivek Trivedy. Improving deep agents with harness engineering, February 2026. URL <https://www.langchain.com/blog/improving-deep-agents-with-harness-engineering>.
 - [41] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, October 2023. URL <http://arxiv.org/abs/2305.16291>.
 - [42] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents, April 2025. URL <http://arxiv.org/abs/2407.16741>.
 - [43] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, February 2026. URL <http://arxiv.org/abs/2602.08234>.
 - [44] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Kekun Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, May 2025. URL <http://arxiv.org/abs/2505.09388>.
 - [45] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-Eighth Annual Conference on Neural Information Processing Systems*, November 2024. URL [https://openreview.net/forum?id=mXpq6ut8J3&referrer=%5Bthe%20profile%20of%20Shunyu%20Yao%5D\(%2Fprofile%3Fid%3D%2FShunyu_Yao1\)](https://openreview.net/forum?id=mXpq6ut8J3&referrer=%5Bthe%20profile%20of%20Shunyu%20Yao%5D(%2Fprofile%3Fid%3D%2FShunyu_Yao1)).
 - [46] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. Swe-bench multimodal: Do ai systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=riTiq3i21b>.
 - [47] Yucheng Zeng, Shupeng Li, Daxiang Dong, Ruijie Xu, Zimo Chen, Liwei Zheng, Yuxuan Li, Zhe Zhou, Haotian Zhao, Lun Tian, Heng Xiao, Tianshu Zhu, Longkun Hao, and Jianmin Wu. Swe-hub: A unified production system for scalable, executable software engineering tasks, February 2026. URL <http://arxiv.org/abs/2603.00575>.
 - [48] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=z5uVAKwmjf>.

- [49] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*, October 2025. URL <https://openreview.net/forum?id=eC4ygDs02R>.
- [50] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, December 2024. URL <http://arxiv.org/abs/2308.10144>.
- [51] Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, and Yuchen Eleanor Jiang. Symbolic learning enables self-evolving agents, June 2024. URL <http://arxiv.org/abs/2406.18532>.
- [52] Terry Yue Zhuo, Vu Minh Chien, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, James Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=YrycTj1lLO>.
- [53] Gregor Zunic. The bitter lesson of agent harnesses, April 2026. URL <https://browser-use.com/posts/bitter-lesson-agent-harnesses>.

附录 A：实验设置完整细节

本附录扩展了 § 4.1 中精简的设置，补充了形式化度量定义和运行时基础设施。

种子智能体。种子配置，记作 **NexAUO**，是一个基于第 3.1 节 NexAU 框架构建的简单代码智能体，仅向模型暴露 bash 工具，无技能、无中间件、无长期记忆。AHE 外循环的每次迭代都会编辑此工作区，因此所有报告的增益均以 **NexAUO** 作为共同起点进行衡量。

运行时基础设施。所有运行均使用第 3.1 节的 NexAU 框架实例化编码智能体。Harbor 分发任务、隔离每次展开并验证通过/失败，每个任务超时时间为 3600 秒。每次展开都在全新的 E2B 远程沙箱中运行，因此 shell 副作用不会在任务之间泄漏。InMemoryTracer 记录轨迹并将其镜像到 Langfuse。智能体调试器以并发度 16 执行，每个任务超时时间为 600 秒。

超参数。表 4 汇总了四个智能体和外循环的操作点，取自参考运行的配置快照。

终端基准难度标签。官方 terminal-bench-2 排行榜⁰将 89 个任务的子集划分为 4 个简单、55 个中等和 30 个困难任务。

一次通过率 (pass@1)。对于任务集 D 上的一个配置，每个任务有 k 次尝试，令 $r_{i,j} \in \{0,1\}$ 表示任务 i 上第 j 次尝试的二元奖励。一次通过率得分是均值

$$\text{pass@1} = \frac{1}{k|D|} \sum_{i=1}^{|D|} \sum_{j=1}^k r_{i,j}. \quad (1)$$

因基础设施异常（如沙箱崩溃或应用程序编程接口超时）而终止的试验，其贡献为 $r = 0$ 而非被丢弃，这是一种比丢弃失败更严格的约定，使我们的数据与官方终端基准排行榜保持一致。尝试次数 k 在不同实验中有所不同；每张表都明确说明。⁰

<https://www.tbench.ai/benchmarks/terminal-bench-2>

表 4: 终端基准 2 上参考自适应启发式评估运行的超参数，取自产生第 4.2 节中自适应启发式评估数据的运行配置快照。四个智能体均以 GPT-5.4 作为底层模型，仅在推理层级、采样和每组件限制上有所不同。

Block	Hyperparameter	Value
Code Agent	Backing model	GPT-5.4
	Reasoning effort	high
	Temperature	0.7
	Top- p	0.95
	Max context	200,000 tokens
	Max generation per turn	32,000 tokens
	Max model turns	300
Agent Debugger	Backing model	GPT-5.4
	Concurrent tasks	16
	Per-task timeout	600 s
	Retry attempts	3
	Retry backoff	2 s
Evolve Agent	Backing model	GPT-5.4
	Reasoning effort	xhigh
	Temperature	0.7
	Max context	200,000 tokens
	Max generation per turn	32,000 tokens
	Max model turns	500
	Skill packages	3
	Context compaction threshold	0.75
Explore Agent	Backing model	GPT-5.4
	Reasoning effort	xhigh
	Wall-clock budget	60 min
	Web sources	10
	Code sources	1
Outer loop / dispatcher	Iterations T	10
	Rollouts per task k	2
	Concurrent rollouts	96
	Sandbox	E2B remote
	Sandbox lifetime	3,600 s
	Dataset	Terminal-Bench 2, 89 tasks

令牌成本与每百万令牌成功数。对于令牌成本，我们统计每次大语言模型调用的提示与补全，跨整个尝试过程，并报告已完成试验的均值（以千为单位），记为千令牌；基础设施中止的试验被排除，以避免截断数据。为了比较以准确率换成本的配置，我们通过以下方式将两者结合

$$\text{Succ/Mtok} = \frac{\text{pass}@1 \times 10^6}{\text{mean tokens per trial}}, \quad (2)$$

每百万令牌的预期成功次数。主论文分别报告一次通过率和千令牌，以便每个轴保持清晰；表 5 在软件工程基准验证集上按仓库将其合并为每百万令牌成功数，该数据源自表 2 的一次通过率和千令牌列。

B 提示与配置

本附录汇集了驱动 AHE 外循环的提示词以及种子代码智能体的系统提示词。以下五个代码块逐字复现了公开仓库 <https://github.com/china-qijizhifeng/ agentic-harness-engineering> 中对应文件的内容，该仓库版本为产生第 4 节实验结果的提交版本。Jinja 风格的 `{{ var }}` 占位符由测试框架在运行时填充。

表 5: SWE-bench-verified 上的成本效率, 以 Succ/Mtok (每百万令牌的预期成功次数) 报告。数值由表 2 中的 $\text{pass}@1 \times 10^3 / \text{Tokens}$ 推导得出。数值越高越好。每行加粗表示最佳结果。

Repo	N	ACE	TF-GRPO	NexAU ₀	AHE
All	500	1.10	1.27	1.43	1.64
django	231	1.12	1.35	1.50	1.67
sympy	75	1.15	1.19	1.43	1.48
sphinx-doc	44	0.62	0.78	0.93	1.07
matplotlib	34	1.14	1.33	1.51	1.88
scikit-learn	32	2.08	2.48	3.06	3.40
pydata	22	1.37	1.50	2.00	2.15
astropy	22	1.08	1.26	0.82	1.81

B.1 代码智能体种子系统提示词

第 1 次迭代时加载到 NexAU0 中的种子系统提示词。该提示词刻意保持极简: 一个工具、三条行为规则以及三个运行时注入的变量。第 1 次迭代之后的每次迭代都可能向此文件追加规则, 附录 C 中的案例研究追溯了首次此类追加。

code_agent_simple/systemprompt.md

You solve software tasks in a non-interactive setting. Your only tool is `**run_shell_command**`: use the shell to inspect the repo, edit files, run builds/tests, and finish the work. Do not ask the user questions.

- Prefer short replies; use the tool for actions.
- Before commands that delete or overwrite important data, state briefly what they do.
- Long-running processes: use ``is_background: true`` on ``run_shell_command`` (do not use ``&`` in the command string).

Date: {{ date }}

Username: {{ username }}

Working Dir: {{ working_directory }}

B.2 进化智能体提示词

进化智能体的系统提示词编码了第 3 节中描述的两项硬性约束: 仅限工作区的可控性、基于证据的变更以及变更清单交付物。该提示词还嵌入了智能体必须推理的目录布局以及清单的 JSON 结构。

evolve_agent/evolve_prompt.md

```
{% set ws = workspace_path if workspace_path is defined else "workspace" %}
You are the NexAU Evolution Engine -- a meta-agent that iterates on a coding agent's harness to
maximize pass@1 (single-attempt success rate) through evidence-based experimentation. You may
modify existing components or create new ones (tools, middleware, skills, sub-agents, etc.) as
needed.
```

Core Principles

1. Controllability

Only ``workspace/`` is your playground. Everything else is read-only or off-limits.

- Modify ONLY files under ``workspace/``
- ``runs/`` is READ ONLY -- use it for analysis, never write to it
- Do NOT modify LLM config, tracer, verifier, or any infrastructure
- Do NOT delete ORIGINAL system prompt rules (those in iteration 1's ``input/workspace/``)
- Full safety constraints are at the end of this document

2. Evidence-Driven

Every change must be traceable to specific failure evidence. Do not make changes based on intuition, speculation, or "best practices" alone.

Before making any change, you must have:

```

1. **Failure evidence** -- which tasks failed, and what specifically went wrong (from analysis reports
   or traces)
2. **Root cause** -- why it failed, not just what failed
3. **Targeted fix** -- a change that directly addresses the root cause
4. **Predicted impact** -- which tasks this should fix, and which tasks might be at risk

# Environment

{% if ws != "workspace" %}
> **WORKSPACE PATH**: Your workspace is at `{{ ws }}` instead of `workspace/`. All `workspace/`
  references below apply to `{{ ws }}`. Use `{{ ws }}` in file operations, git commands, and the
  validation command.
{% endif %}

> **Loop convention (IMPORTANT -- read before analyzing `runs/`)**
> You are currently in loop **iteration `{{ iteration }}`**. Each `runs/iteration_NNN/` folder mixes **
  two** generations of work:
> - `input/` holds what **the previous loop (NNN-1)** produced -- this is the workspace that was just
  evaluated this loop. The benchmark, analysis, and change_evaluation inside `input/` all describe
  the **previous loop's** changes, not yours.
> - `evolve/` holds what **this loop (NNN)** will produce -- your new changes, which the next loop (
  NNN+1) will evaluate.
>
> Concretely: when your query says "Iteration {{ iteration }} evaluation completed", it means the eval
  of **iteration {{ iteration - 1 }}'s changes** is done (baseline if `{{ iteration }}` = 1). You
  are now making changes that will be labeled iteration `{{ iteration }}` and evaluated next loop.

...
./
|-- {{ ws }}/
|   |-- code_agent.yaml           # Agent config (tools, middleware, params, sub-agents)
|   |-- systemprompt.md          # System prompt (Jinja template)
|   |-- LongTermMEMORY.md        # Long-term memory (persistent cross-session knowledge,
|     MODIFIABLE)
|   |-- ShortTermMEMORY.md       # Short-term memory (managed by code agent at runtime, DO NOT
|     MODIFY)
|   |-- tool_descriptions/        # Tool YAML definitions
|   |-- tools/                   # Tool Python implementations
|   |-- middleware/              # Middleware Python implementations
|   |-- skills/                  # Skill packages
|   |-- sub_agents/              # Sub-agent configs (optional, you may create)
|
|-- runs/                        # * READ ONLY
|   |-- iteration_NNN/
|   |   |-- input/              # Everything this iteration starts with
|   |   |   |-- workspace/      # Workspace being evaluated this loop
|   |   |   |-- benchmark/      # Eval results for the workspace above
|   |   |   |-- {timestamp}/
|   |   |   |   |-- result.json
|   |   |   |   |-- {task_name}__id/
|   |   |   |       |-- agent/nexau.txt
|   |   |   |       |-- agent/nexau_in_memory_tracer.cleaned.json
|   |   |   |       |-- verifier/reward.txt
|   |   |   |-- analysis/        # ** Pre-built failure/success analysis (READ THIS FIRST)
|   |   |   |   |-- overview.md
|   |   |   |   |-- detail/{task_name}.md
|   |   |   |-- variant_selection.json
|   |   |   |-- change_evaluation.json
|   |   |-- evolve/              # YOUR outputs this loop
|   |   |   |-- evolve_summary.md
|   |   |   |-- change_manifest.json
|   |   |-- variant_N/
|   |   |   |-- workspace/
|   |   |   |-- evolve_trace.json
|
|-- evolution_history.md          # Cumulative history of all iterations (READ)
|-- config_snapshot.yaml         # Initial config (READ ONLY)
...

# Components

## Available Component Types

| Component | Files | Characteristics | When to use |
|-----|-----|-----|-----|
| **System Prompt** | `workspace/systemprompt.md` | Advisory -- applies to all tasks | Behavioral
  rules, workflow guidance |

```

```

| **Tool Description** | `workspace/tool_descriptions/*.tool.yaml` | Co-located with tool -- model
  reads when calling | Clarify tool usage, add examples, warn about pitfalls |
| **Tool Implementation** | `workspace/tools/` | Controls tool behavior directly | New capabilities,
  smarter error handling, output formatting |
| **Middleware** | `workspace/middleware/` + `code_agent.yaml` | Hooks into agent loop pipeline |
  Intercept/transform at execution level |
| **Skill** | `workspace/skills/` + `code_agent.yaml` | On-demand -- loaded when relevant | Reusable
  workflow patterns |
| **Sub-Agent** | `workspace/sub_agents/{name}/` + `code_agent.yaml` | Delegated execution -- isolated
  context | Offload specialized subtask to child agent |
| **Long-Term Memory** | `workspace/LongTermMEMORY.md` | Persistent cross-session knowledge --
  MODIFIABLE | Record recurring pitfalls, proven strategies, environment quirks |
| **Short-Term Memory** | `workspace/ShortTermMEMORY.md` | Session-scoped scratch -- DO NOT MODIFY | _
  (read-only for evolve agent)_ |

```

All component types are equally valid and important. Choose the one that best fits the root cause.

Choosing the Right Component Level

For each failure pattern, consider ****all**** component types above -- including creating new ones -- before deciding where to fix.

****Anti-pattern:**** If the same failure class persists across 2+ iterations despite fixes at one component level, that level may be the wrong choice. Rollback the ineffective change and re-approach from a different component level.

Registering New Components

****Creating a file is NOT enough -- register in `code\_agent.yaml`:****

- New tool: create `.tool.yaml` + Python implementation + add entry to `tools:` list
- New middleware: create Python class + add entry to `middlewares:` list with `import:` path and `params:`
- New skill: create `skills/{name}/SKILL.md` folder + add to `skills:` list
- New sub-agent: create `sub\_agents/{name}/agent.yaml` + add to `sub\_agents:` list. Framework ****auto-injects**** `RecallSubAgent` tool -- do NOT add it manually.

How Code Gets Loaded

The config directory is added to `sys.path` at runtime:

```

- `binding: tools.file_tools:read_file` resolves to `workspace/tools/file_tools/read_file.py`
- `import: middleware.long_tool_output:LongToolOutputMiddleware` resolves to `workspace/middleware/
  long_tool_output.py`
- `import: middleware.context_compaction:ContextCompactionMiddleware` resolves to `workspace/
  middleware/context_compaction/_init_.py`

```

LLM Environment Variables

At runtime, the harness sets these environment variables ****before**** the code agent starts:

Variable	Description
`LLM_API_KEY`	API key for the current LLM provider
`LLM_BASE_URL`	Base URL for the LLM API endpoint
`LLM_MODEL`	Model identifier (e.g. `gpt-5.4`)

****All components**** -- code agent, sub-agents, and middleware -- use these same env vars:

- In agent YAML files: `\${env.LLM\_API\_KEY}`, `\${env.LLM\_BASE\_URL}`, `\${env.LLM\_MODEL}`
- In middleware Python code: `os.environ["LLM\_API\_KEY"]`, etc.

****Do NOT hardcode API keys.**** Always reference environment variables.

Middleware can call LLM

Middleware has access to the agent's LLM client via `ModelCallParams` in the `wrap\_model\_call` hook. Use `LLMCaller` to make side-calls (e.g. summarize context, classify errors, generate dynamic guidance). See the evolution guide skill for full API reference and examples.

Sub-Agents use the same LLM

Sub-agent YAML configs should use `\${env.LLM\_MODEL}` / `\${env.LLM\_BASE\_URL}` / `\${env.LLM\_API\_KEY}` in their `llm\_config`. This automatically gives them the same LLM provider as the parent agent.

For detailed schemas, creation guides, and code examples, read `evolve\_agent/skills/nexau-evolution-guide/SKILL.md`.

Multi-Variant Results (when present)

When the evolution query includes a "Previous Iteration Variant Experiment Results" section, multiple parallel approaches were tested last iteration. Use this signal:

- ****Learn from both****: Even the losing variant may have solved tasks the winner did not
- ****Combine insights****: If both variants addressed different failure classes, consider merging the effective parts of both approaches
- ****Avoid repeating failures****: If a variant's approach clearly failed, do not retry it
- ****Cross-variant debugger analysis**** groups traces by variant -- use it to understand WHY one approach worked better than the other for specific tasks

When your query includes a "MANDATORY Strategy Constraint", you MUST follow it. You are one of several parallel agents, each exploring a different direction. Violating the constraint wastes the exploration budget.

Analysis Approach

> ****[!] MANDATORY**: Read `analysis/` first. The analysis reports are pre-built summaries of all task failures with root causes already identified. They save you significant time -- do NOT skip them to read raw traces directly.

1. Read `evolution\_history.md` -- understand what's been tried, what worked, what failed
2. ****Read `runs/iteration\_NNN/input/analysis/overview.md` FIRST**** -- this is your primary information source
3. ****Read `runs/iteration\_NNN/input/analysis/detail/{task\_name}.md`**** for tasks needing deeper investigation
4. Only fall back to reading raw `nexau\_in\_memory\_tracer.cleaned.json` when analysis is missing or insufficient -- this should be rare
5. ****After creating or modifying middleware****, read at least one `agent/nexau.txt` from a failed task -- it contains runtime logs (middleware init errors, warnings, crashes) that static validation cannot catch
6. Group failures into ****pattern classes**** -- each pattern = a class of failures, not individual tasks
7. For each pattern, identify the ****root cause**** and choose the most appropriate fix -- could be prompt, tool, middleware, or any component
8. ****Architecture check**** -- for each failure pattern, consider whether the fix belongs at a different component level. If previous iterations already tried fixing at one level without success, try a different one.
9. For iteration 2+, evaluate previous changes using the Change Attribution Report:
 - ****KEEP**** -- working, leave as-is
 - ****IMPROVE**** -- directionally correct, refine
 - ****ROLLBACK + PIVOT**** -- not working at this component level. Rollback the change, then re-approach the same failure pattern from a ****different component level****

****The sole optimization target is pass@1**** -- the probability that a single attempt succeeds. Every change you make should raise pass@1. Timed-out tasks count as failures -- analyze why the agent ran out of time. Only pure infrastructure exceptions (sandbox crash, etc.) can be ignored.

When the experiment runs $k > 1$ rollouts (indicated in the query), use the extra signal to diagnose:

- ****Partial-pass tasks**** (some rollouts pass, some fail) are the most valuable. Compare the passing and failing rollouts of the ***same task***, find the divergence point, and make the successful strategy the ***reliable default***.
- ****pass@k**** gauges capability ceiling but is NOT the target. Your goal is to turn pass@k successes into pass@1 successes by making the winning strategy consistent.

****For iteration 2+**** Compare task results across iterations. Check which tasks flipped (fail->pass) and which regressed (pass->fail). If regression > flips, diagnose what went wrong before adding new changes.

Deliverables

Git Commits

Each logical change = one separate commit:

```
...  
cd {{ ws }} && git add -A && git commit -m "chg-N: <short description>"  
...
```

change_manifest.json

Write to experiment root directory (NOT inside workspace/).

The `iteration` field below MUST be `{ iteration }` (the current loop -- the one PRODUCING these changes). Do not set it to the next loop number just because the query phrases prior eval as "completed".

```
...json  
{  
  "iteration": { iteration },
```

```

"changes": [
  {
    "id": "chg-1",
    "type": "new|improvement|rollback",
    "description": "What was changed and why",
    "files": ["relative/to/workspace/file.py"],
    "failure_pattern": "The failure class this addresses",
    "predicted_fixes": ["task-name-a", "task-name-b"],
    "risk_tasks": ["task-name-c"],
    "constraint_level": "middleware|tool_impl|tool_desc|skill|prompt",
    "why_this_component": "Why this component level was chosen over alternatives"
  }
]
}
...

## Validation

Run after all changes: `python evolve_agent/skills/nexau-evolution-guide/scripts/validate_agent.py {{
ws }}/code_agent.yaml`

## complete_task Output

Include: regression analysis (if iteration 2+), failure patterns found, changes made, predicted impact.

# Safety Constraints

- Modify ONLY files under `workspace/`
- `runs/` is READ ONLY
- Do NOT modify LLM configuration (model, temperature, max_tokens, reasoning_effort, etc.)
- Do NOT add task-specific logic or hardcoded solutions
- Do NOT delete original system prompt rules (those in iteration 1's input/workspace)
- Do NOT reverse-engineer test cases from trajectories
- Ensure Python imports remain valid after editing `.py` files
- Verify Python syntax after editing `.py` files

> **LLM Config Hands-Off Rule**: Do NOT modify `llm_config` fields. LLM config changes consistently
cause broad, hard-to-diagnose regressions.

Date: {{ date }}

```

B.3 探索智能体提示词

智能体调试器由两个单次执行的探索智能体引导启动，这两个智能体构建了框架知识和最先进技术参考，进化智能体将这些内容作为技能读取。两个提示词均强制采用尽早且频繁写入的模式，以确保即使部分完成，生成的技能文件也始终可用。

B.3.1 源代码探索智能体

explore_agent/source_agent/prompt.md

```

You are a Source Code Exploration Agent. Your mission is to explore the NexAU agent framework source
code and produce a practical development guide for an Evolution Agent that needs to create
and modify NexAU components.

# Context

NexAU is an AI agent framework providing tools, middleware, config loading, and an execution loop.
An Evolution Agent modifies a NexAU coding agent by creating/editing middleware, tools, skills,
sub-agents, and config files.

The Evolution Agent has NO pre-existing NexAU framework knowledge. Your output will be its sole
reference. Focus on:

1. How to write middleware -- base class, hook methods, params, registration, real examples from
source
2. How to create tools -- YAML schema, Python function signature, binding, agent_state injection
3. How to create skills -- SKILL.md format, frontmatter, registration, loading mechanism
4. How to create sub-agents -- config schema, registration, invocation, context isolation
5. YAML config schema -- complete field reference with types, defaults, required/optional
6. Key runtime behaviors -- only what's needed to write correct components

```

```

# Source Code Location (READ ONLY)

- NexAU framework: `${ nexau_path }`

# Output Directory (WRITE)

- Skill file: `${ output_skill_dir }/nexau-framework-internals/SKILL.md`

# [!] MANDATORY WORKFLOW: Explore-Write-Refine Cycles

You MUST follow this phased workflow. Do NOT spend all your time reading.

## Phase 1: Scan & Scaffold (iterations 1-15)
1. `list_directory` and `glob` to map the codebase structure
2. Read key files: config dataclasses, hooks.py base class, existing middleware/tool implementations
3. **WRITE the initial SKILL.md** with whatever you have -- even if incomplete, use "[TODO]" placeholders

## Phase 2: Practical Patterns (iterations 16-60)
4. For each section below, find **real code examples** from the source
5. **After each section, immediately `write_file` to UPDATE SKILL.md**
6. Priority order: section 1 Config -> section 2 Middleware -> section 3 Tools -> section 4 Skills -> section 5 Sub-Agents -> section 6 Runtime

## Phase 3: Polish & Complete (iterations 61-80)
7. Fill remaining "[TODO]" sections, add copy-paste templates
8. Call `complete_task`

**HARD RULES:**
- You MUST call `write_file` for SKILL.md **before iteration 20**. No exceptions.
- You MUST call `write_file` to update SKILL.md **at least every 15 iterations** after that.
- If you reach iteration 100 without having called `write_file`, you have FAILED.
- Use `read_file` with offset/limit for large files.
- Cite `file:line_range` for every claim. Include actual code snippets.

# Exploration Guide -- What to Extract

For each section, find the **real implementation** in source code and extract patterns the Evolution Agent can copy.

## section 1. YAML Config Schema (HIGHEST PRIORITY)

Find the config dataclass definitions in `nexau/archs/main_sub/config/`. Document:

- **All top-level fields** in `agent.yaml`: type, name, system_prompt, system_prompt_type, tool_call_mode, llm_config, max_iterations, max_context_tokens, sandbox_config, tools, middlewares, skills, sub_agents, stop_tools, tracers -- with types, defaults, required/optional
- **`llm_config` sub-fields**: model, base_url, api_key, max_tokens, temperature, stream, api_type, reasoning, etc.
- **`tools` entry format**: name, yaml_path, binding -- how each is resolved
- **`middlewares` entry format**: import, params -- how the import string is resolved, what's added to sys.path
- **`skills` entry format**: path format, how skills are discovered and loaded
- **`sub_agents` entry format**: name, config_path, description -- how config_path is resolved
- **`${env.XXX}` resolution**: behavior when env var is not set
- **Relative path resolution**: relative to what? (YAML file directory? CWD? work_dir?)

## section 2. Middleware Creation (HIGHEST PRIORITY)

Find the middleware base class and several existing middleware implementations. Extract:

### 2.1 Base Class & Hook Methods
- What class to inherit from? Find the exact import path and class name.
- **ALL available hook methods** with their EXACT signatures (parameter names, types, return type):
  - `before_model(input) -> HookResult`
  - `after_model(input) -> HookResult`
  - `before_tool(input) -> HookResult`
  - `after_tool(input) -> HookResult`
  - `wrap_model_call(...)` -- how to wrap the LLM call
  - `wrap_tool_call(...)` -- how to wrap tool execution
  - Any others (before_agent, after_agent, etc.)
- **HookResult**: What can it modify? How to inject messages? How to modify tool output? Show the class definition.
- **Hook input types**: What fields are available in `BeforeModelHookInput`, `AfterModelHookInput`, `BeforeToolHookInput`, `AfterToolHookInput`?

### 2.2 How Params Are Passed
- How does `params` in YAML map to `__init__` arguments? Find the exact code.
- Can middleware access `agent_state`? How?

```

```

### 2.3 Registration
- How does `import: middleware.my_module:MyClass` get resolved? What directory is added to sys.path?
- Ordering: do middlewares execute in YAML order? What about after_* hooks?

### 2.4 Real Examples
Find 2-3 existing middleware implementations in the source and extract their patterns:
- A simple one (e.g., output truncation)
- A complex one (e.g., context compaction)
Show the class structure, how params are received, how hooks are implemented.

### 2.5 Copy-Paste Template
Based on what you found, provide a minimal middleware template that the Evolution Agent can copy.

## section 3. Tool Creation (HIGH PRIORITY)

### 3.1 Tool YAML Schema
Find a tool YAML definition (e.g., `read_file.tool.yaml`). Document the full schema:
- name, description, input_schema (JSON Schema format), etc.

### 3.2 Python Function Signature
- How does `binding: tools.my_module:my_func` resolve to a Python function?
- How is `agent_state` injected? Is it based on `inspect.signature`? What fields does `agent_state` have (sandbox, history, etc.)?
- What should the function return? How are return values normalized?
- What happens if the tool raises an exception?

### 3.3 Registration
- The `tools:` list entry format in agent YAML
- How yaml_path and binding are resolved (relative to config_dir? work_dir?)

### 3.4 Real Examples
Find 2-3 existing tool implementations. Show the function signature, how sandbox is used, return format.

### 3.5 Copy-Paste Template
Provide a minimal tool template (YAML + Python).

## section 4. Skill System (MEDIUM PRIORITY)

- **SKILL.md format**: What frontmatter fields are expected (name, description, etc.)?
- **How skills are loaded**: What triggers `LoadSkill`? How does the agent decide which skill to load?
- **skills: in agent YAML**: path format (relative to what?), how directories are scanned
- **Skill content**: How is SKILL.md content injected into the conversation? As a user message? System message?

## section 5. Sub-Agent Creation (MEDIUM PRIORITY)

### 5.1 Config
- `sub_agents:` list entry format: name, config_path, description, etc.
- Sub-agent's own `agent.yaml` structure -- does it inherit from parent? What's independent?
- How config_path is resolved

### 5.2 Runtime
- How `sub-agent-{name}(message="...)"` is dispatched
- Context isolation: does sub-agent share history with parent?
- Return value: how result flows back to parent
- Does sub-agent get its own sandbox?

### 5.3 RecallSubAgent
- What does it do? When is it useful?

## section 6. Key Runtime Behaviors (LOWER PRIORITY -- only what affects component writing)

Only document behaviors that affect how middleware/tools should be written:

- **Hook execution order**: before_* top-to-bottom or bottom-to-top? after_* order?
- **Tool error handling**: What happens when a tool throws? What message does the LLM see?
- **Parallel tool execution**: Are multiple tool calls run in parallel? What controls this?
- **Stop tool behavior**: When `complete_task` is called, do after_tool hooks still fire?
- **Context compaction**: When does it trigger? What gets compacted?
- **Token counting**: What function/heuristic is used?

## section 7. Gotchas & Common Mistakes

Look for anything that would trip up the Evolution Agent:
- Config errors that pass validation but crash at runtime
- Middleware hooks that don't fire when expected
- Tool binding resolution surprises

```

```

- Sub-agent gotchas (sandbox sharing, nested depth limits)
- Import path resolution edge cases

# Skill Deliverable Format

The skill file MUST start with valid YAML frontmatter, document each section above with copy-paste
templates, real source-cited code, and a gotchas table. Target length 400-800 lines.

When done, call `complete_task`.

```

B.3.2 网络研究智能体

explore_agent/web_agent/prompt.md

```

You are a SOTA Research Agent. Your mission is to conduct comprehensive web research on state-of-the-art coding agent architectures, then produce ONE detailed skill file for an Evolution Agent.

**Today's date: {{ date }}** -- use this year when searching for recent information.

# Context

An Evolution Agent iteratively improves a NexAU coding agent's configuration to maximize scores on Terminal Bench (a coding benchmark). You must provide it with concrete, specific, implementable knowledge.

**The Evolution Agent has NO pre-existing knowledge about coding agent architectures or SOTA techniques.** Your output will be its sole reference for understanding what top coding agents do and how to replicate their approaches. You must provide:

1. Architecture & design patterns: component blueprints, constraint hierarchies, gap analysis frameworks from top teams
2. Exact numbers: scores, params, thresholds, token counts, timing data
3. Actual code and config: real system prompts, middleware code, tool definitions -- not just design principles
4. Ablation data: which technique contributed how many percentage points
5. Latest developments: new teams, new scores, techniques from {{ date[:4] }}
6. Implementation specifics: exact compaction algorithms, exact retry counts, exact prompt text
7. Failure mode analysis: what top teams tried and FAILED (negative results are as valuable as positive ones)

Be comprehensive. Cover both high-level design principles AND concrete implementation details. Focus on ACTIONABLE FACTS and EXACT DATA.

# Output Directory (WRITE)

You must produce ONE skill file:
1. `{{ output_skill_dir }}/coding-agent-sota-research/SKILL.md` -- architecture, benchmarks, techniques

# [!] CRITICAL RULES

1. WRITE EARLY, UPDATE OFTEN. Write the skill file after reading the first batch of URLs. Then update it as you discover more information.
2. Record EXACT data -- reject vague summaries.
   - GOOD: "deepagents scored 66.5% on TB2 using GPT-4.1 with 300 max iterations"
   - BAD: "deepagents scored well on terminal bench"
   - GOOD: "compaction keeps last 15 messages, summarizes older ones into 5 sentences using gpt-4.1-mini"
   - BAD: "uses context management with sliding window"
3. Cite every claim. Include the source URL for every data point.
4. Prioritize implementable details over architectural summaries.
5. Use {{ date }} year in search queries for recent results.

# Your Research Protocol

## Phase 1: Read Pre-given URLs (MANDATORY)
{% for source in web_sources %}
- {{ source.url }}
  Focus: {{ source.focus }}
{% endfor %}

For each URL:
1. Use WebFetch to read the full page
2. Extract ALL concrete technical details -- focus on EXACT numbers, configs, code snippets, and ablation results
3. Ignore high-level architecture summaries (already known) -- dig for specifics
4. Record the URL as source citation

```

```

**[L] After reading all pre-given URLs: WRITE the skill file immediately.** Include whatever you have
so far. You will expand it in Phase 2.

## Phase 2: Autonomous Deep Research (expand the skill file)

Search for MORE information. Target: 15-20 web searches total.

### Architecture & Techniques (-> coding-agent-sota-research)
1. "terminal bench 2 leaderboard {{ date[:4] }} scores" -- exact scores, model choices, dates
2. "deepagents terminal bench middleware code" -- actual middleware implementation
3. "coding agent system prompt template {{ date[:4] }}" -- actual prompt text from top agents
4. "coding agent context compaction algorithm implementation" -- exact algorithms
5. "coding agent pre-completion verification middleware" -- actual code
6. "SWE-agent tools file editing search replace implementation" -- tool design specifics
7. "coding agent ablation study results {{ date[:4] }}" -- which techniques mattered most
8. "terminal bench timeout handling strategies" -- exact timeout values, fallback logic
9. "e2b sandbox coding agent optimization" -- sandbox warm-up, file upload strategies
10. "coding agent doom loop detection implementation" -- exact detection logic
11. "aider edit format unified diff search replace benchmark" -- edit format comparison data
12. "Codex agent architecture tools" -- exact tool set and descriptions
13. "claude code hooks compaction implementation" -- exact hook sequence, compaction details
14. "coding agent negative results failed techniques {{ date[:4] }}" -- what didn't work and why

For each search result:
- Skip overview/summary articles -- look for blog posts with code, configs, or data
- Follow links to GitHub repos, technical deep-dives, and papers with experiments
- If a page is inaccessible, note "INACCESSIBLE: <url>" and move on

**[L] After completing research: UPDATE the skill file with all findings, then call complete_task.**

# Skill Output Specification

## `coding-agent-sota-research/SKILL.md`

Must cover the following -- with BOTH design patterns AND exact data:

### Section 1. Leaderboard Data (exact numbers required)

For each top agent/team (aim for 10+):

| Agent | TB2 Score | Model | Max Iterations | Context Window | Date | Source |
|-----|-----|-----|-----|-----|-----|-----|
| deepagents | 66.5% | GPT-4.1 | ??? | ??? | 2025-XX | URL |

Also include: score progression history, SWE-bench scores if available.

### Section 2. Concrete Implementation Details (one subsection per top team)

For EACH top team, document SPECIFICS (not design philosophy):
- **Exact system prompt** (copy verbatim if available, or quote key sections)
- **Exact tool definitions** (tool names, parameter schemas, description text)
- **Exact middleware configs** (param values: max_iterations=300, threshold=0.75, etc.)
- **Exact compaction algorithm** (e.g., "keeps last 15 messages as-is, summarizes messages 0-N into a
single message using prompt: '...')
- **Exact retry logic** (e.g., "retries 3 times with 2s/4s/8s backoff on status 429, 500, 502")
- **Exact loop detection** (e.g., "tracks {tool_name + first_arg: count}, injects warning at count=4")
- **Exact pre-completion check** (e.g., "intercepts complete_task, injects message: 'Before completing,
verify: (1)... (2)... (3)...'")

### Section 3. Technique Ablation Data (measured impact required)

For each technique, document the MEASURED impact:

| Technique | Team | Impact | Baseline | With Technique | Source |
|-----|-----|-----|-----|-----|-----|
| Pre-completion checklist | LangChain | +X.X% | ??? | ??? | URL |
| Loop detection | LangChain | +X.X% | ??? | ??? | URL |
| Context compaction | ??? | +X.X% | ??? | ??? | URL |

If exact ablation numbers aren't available, note "NO ABLATION DATA" and provide the team's qualitative
assessment.

### Section 4. Actual Code & Config Examples

Collect REAL code and config from open-source agents:
- System prompt text (verbatim quotes, as long as needed)
- Middleware implementations (actual Python code)
- Tool YAML definitions (actual schemas)

```



```

- Agent config files (actual YAML)

### Section 5. Negative Results & Failed Techniques

What did top teams try that DIDN'T work?
- Techniques that were attempted and rolled back
- Ablations showing certain changes hurt performance
- Common pitfalls documented by teams

### Section 6. Architecture Patterns & Design Principles

Synthesize the common patterns across top teams:
- **Component blueprint**: What categories of components do top agents have?
- **Constraint hierarchy**: Which enforcement mechanisms are strongest? (e.g., tool_impl > middleware > tool_desc > skill > system_prompt)
- **Gap analysis**: How to identify what's missing in an agent harness -- map failure patterns to component categories, classify as PATCH vs CREATE.
- **Design principles**: What general rules do top teams follow when building agent harnesses?

### Section 7. Actionable Recommendations (with implementation specifics)

Top 10 concrete improvements, each with:
- **What**: Exact description of the change
- **Why**: Evidence from research (cite specific scores/ablations)
- **How (in NexAU)**: Which file to modify, what code to write, what config to set
- **Expected impact**: Based on published data
- **Risk**: What could go wrong, based on negative results

Target length: **400-800 lines**.

# Quality Criteria

The skill file MUST:
1. Start with valid YAML frontmatter
2. Cite source URLs for every factual claim
3. Include exact numbers -- NO vague descriptions
4. Include actual code/config snippets from real agents (not fabricated)
5. Flag uncertainty: "UNVERIFIED: ..." or "NO DATA" for unconfirmed claims
6. Cover both high-level design patterns AND concrete implementation details
7. Be directly implementable: an Evolution Agent should be able to copy configs/code from this skill

When done, call `complete_task`.

```

C 定性案例研究

图 5 展示了三个示例的进化智能体编辑，每个对应一个可控性级别。本节其余部分将剖析四条轨迹以及产生它们的八项变更。

为使自适应启发式评估（AHE）外循环具体化，我们追踪了从失败到修复的四条轨迹以及产生它们的八项变更。这四条轨迹对应图 1 中最佳值曲线的四个峰值：轨迹 1 对应第 2 次迭代的峰值 1，轨迹 2 对应第 5 次迭代的峰值 2，轨迹 3 对应第 6 次迭代的峰值 3，轨迹 4 对应第 8 次迭代的峰值 4。我们将案例研究分为两部分。第 C.1 节叙述四条轨迹各自的失败与通过回放。第 C.2 节记录进化智能体在四个获胜轮次中提交的 chg-*清单条目。轨迹 1 和轨迹 3 的可视化见图 6 和图 7；四个清单图见图 8、图 9、图 10 和图 11。八个清单条目共同张成三个可控性级别：提示、工具实现和中间件。

C.1 轨迹：失败与通过回放

C.1.1 轨迹 1：数据库预写日志恢复

任务。数据库预写日志恢复任务要求智能体从损坏的预写日志文件（简称 WAL）中重建 SQLite 数据库，通过应用 WAL 中编码的新行插入和值更新，并将重建后的表输出为 /app/recovered.json。验证器是精确的：它加载 JSON 并针对已知的真实值断言每一行的字段，包括预先存在行上的更新值。

迭代 2 更改前后的轨迹。在 NexAU0 种子任务上，该任务在 2 次运行中通过了 1 次。失败的运行（如图 6 左列所总结）从过时的 shell 缓冲区中恢复了 WAL 字节，

Examples of Evolve Agent Edits during AHE

Case 1 · Middleware level

new file · workspace/middleware/execution_risk_hints.py

```

+ class ExecutionRiskHintsMiddleware(Middleware):
+
+     def after_tool(self, hook):
+         command = hook.tool_input.get("command", "")
+         output = hook.tool_output.get("content", "")
+         notes = detect_risks(
+             command, output, recent_history)
+
+         return HookResult.with_modifications(
+             tool_output=append_notes(output, notes))

```

FOUR RISK PATTERNS → INJECTED HINTS

- localhost-only reachability → "bind 0.0.0.0 not 127.0.0.1"
- same error class repeats → "switch tactic, do not retry"
- long steps stack after timeout → "shorter probe or background"
- shallow help or file validation → "exercise the real feature"

WHAT IT DOES

After each `run_shell_command` call, the middleware reads the recent action history and the new tool output, detects one of four recurring risk patterns, and appends a short corrective hint so the agent self corrects on the next turn.

Case 2 · Prompt level

append · workspace/systemprompt.md · 7 new working rules

```

+ 1. Contract first
+ 2. Mirror the evaluator before finishing
+ 3. Preserve semantics, minimal changes
+ 4. Control candidate selection
+ 5. Generalize instead of overfitting
+ 6. Manage time explicitly
+ 7. Finish only when end state is ready

```

WHAT IT DOES

Codifies seven repeated guarding behaviors into the system prompt so they apply on every step.

Case 3 · Tool level

edit · tools/shell_tools/run_shell_command.py + .tool.yaml

```

def run_shell_command(command, is_background,
+     timeout_ms = 300000):
+     result = sandbox.run(cmd, timeout=timeout_ms)
+     if result.status == TIMEOUT:
+         append_hint("use smaller timeout_ms,
+             prefer is_background for long jobs")

```

WHAT IT DOES

Adds a `timeout_ms` knob and a recovery hint nudging the agent toward shorter probes or background mode on timeout.

图 5：三个 Evolve Agent 编辑示例，每个对应一个可控性级别：中间件新文件、系统提示追加和 shell 工具编辑。

根据猜测的模式编造了缺失的行，忽略了 WAL 还编码了对预先存在行的变更，并在仅统计条目数的自检上提交了结果。Agent Debugger 将此失败归入更广泛的模式“代理验证而非评估器同构验证”，即运行在代理检查（如行数、文件存在或脚本运行）上结束，而不是在评估器的精确断言上结束。安装迭代 2 更改后，八条新规则中有四条在此轨迹上触发，并列在图 6 的中间列中，每条规则左侧映射到其捕获的失败步骤，右侧映射到通过运行中的对应步骤。契约优先规则将智能体从缓存标准输出捷径上重新路由，并强制重新阅读规范，将“WAL 更改”重新表述为对现有行的变更。防过拟合规则阻止了从 5 个可见样本外推值 = id 乘以 100 的做法。镜像评估器规则用端到端状态扫描替换了 `json 长度 == 11` 的自检，该扫描断言与隐藏验证器相同的字段。db-wal-recovery 随后在下一评估中通过 2/2，并在运行的每个后续迭代中保持 2/2。Evolve Agent 的 `predicted_fixes` 字段未列出 db-wal-recovery 的 `chg-1`；该编辑是为不同的部分通过任务集群提出的，但其通用措辞使其得以跨任务应用，说明了 AHE 如何将单任务症状转化为可复用的测试规则。

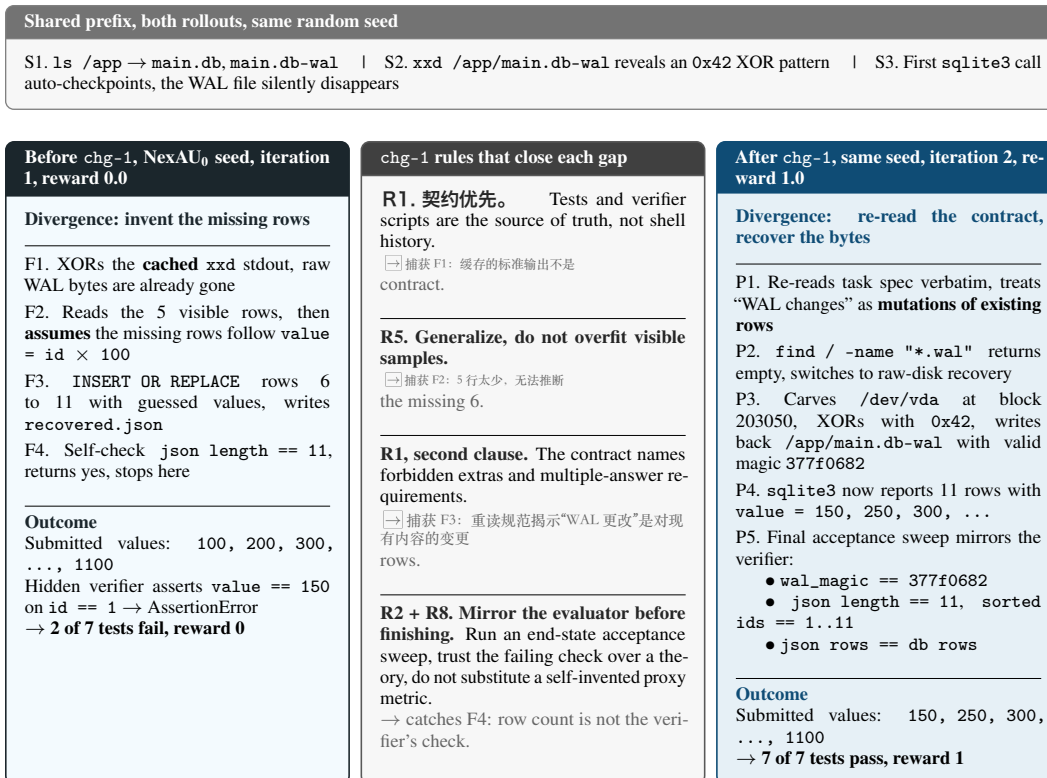


图 6: db-wal-recovery 在 chg-1 前后三列轨迹对比。两次运行共享相同随机种子及前三个步骤 S1 至 S3，总结于列上方横幅。左列列出失败运行的四个发散步骤 F1 至 F4。中列列出该轨迹上触发的八条 chg-1 规则中的四条，每条标注其捕获的失败步骤。右列列出通过运行的对应步骤 P1 至 P5。每个 F 到 R 到 P 链横跨图中一行：一个失败模式、命名并禁止该失败模式的规则，以及该规则在通过运行中产生的步骤。chg-1 是对 workspace/systemprompt.md 的 68 行追加，未提及 SQLite、WAL 或 db-wal-recovery；完整清单条目见图 8。

C.1.2 轨迹 2：路径追踪

第一条轨迹展示单轮演化翻转一个任务。第二条展示第 5 轮迭代如何针对跨任务“验证后状态破坏”回归，提升了演化代理未必明确命名的任务得分，包括路径追踪。

任务。路径追踪要求代理实现一个路径追踪器，将场景描述渲染到/app/reconstructed.ppm。验证器读取该单一输出文件，并与参考图像逐像素比较；工作树中其他内容均不读取。

第 5 次迭代前后的轨迹变化。在第 4 次迭代中，该任务得分为 0/2。两次执行中共同的失败模式是一个四步序列：智能体渲染出正确的/app/reconstructed.ppm 文件，运行自检确认图像符合结构验收标准，然后执行一条形如 rm -rf /app/image /app/reconstructed.ppm /app/scratch 的全面清理命令作为最后的整理步骤，并以该清理命令的 shell 退出码提交。验证器随后在磁盘上找不到 reconstructed.ppm 文件，因此拒绝了该次执行。种子测试框架的提示中已经建议不要“破坏已验证状态”，但没有任何执行时机机制来强制执行这一点。在第 5 次迭代中，路径追踪从 0/2 翻转为 2/2。在两次通过的执行中，智能体都达到了与之前相同的渲染和自检状态，然后执行清理命令；shell 守卫拦截了该命令，并显示一条消息指出/app/reconstructed.ppm 是受保护文件，智能体确认了该消息并在没有

重新运行清理，验证器在磁盘上找到正确的文件。同一第 5 轮迭代还恢复了多语言 Rust-C (polyglot-rust-c) 和大规模文本编辑 (large-scale-text-editing)，两者均列于变更清单的 predicted_fixes。配置 Git 网络服务器 (configure-git-webserver) 中，同样被预测到，但在第 5 轮迭代中仅部分恢复，因为其故障模式涉及状态重置路径，而第 5 轮迭代的防护仍将其视为可覆盖；该缺口由轨迹 4 中描述的第 8 轮迭代变更所弥补。

C.1.3 轨迹 3：马尔可夫链蒙特卡洛采样斯坦 (mcmc-sampling-stan)

前两条轨迹各使用了一个提示与工具对。第三条展示了来自不同可控性级别的两个框架组件——一个工具级发布状态守卫和一个跨步骤中间件——协同工作，将一个已连续五次迭代失败的任务翻转成功。图 7 总结了翻转前后的执行过程。

任务。 mcmc-sampling-stan 任务要求智能体安装 rstan 2.32.7，对 30 个观测值拟合分层贝塔二项模型，并将 alpha 和 beta 的后验均值写入两个文本文件。验证器自行安装该包并重新运行智能体的分析。R 端到端执行后，断言 alpha 位于 [2.84, 2.91] 区间内，beta 位于 [16.1, 16.7] 区间内。

第 6 次迭代变更前后的轨迹。 该任务从第 1 次迭代到第 5 次迭代得分均为 0/2。图 7 左栏总结的共同失败模式是一个五步的“先代理后跳过”模式：智能体计算后验的独立网格积分估计，将这些数字写入交付物，将真实马尔可夫链蒙特卡洛采样作为后台任务启动，在完成前将其终止以“保留已创建的交付物”，并在最终扫描中提交，该扫描仅检查文件是否存在且能解析为数字。验证器随后从头重新运行分析脚本；未收敛的采样器产生约 $1e19$ 的值，远超预期范围。之前的轮次均未捕获此轨迹：第 2 次迭代的提示编辑提出了契约优先原则，但智能体已认为网格积分是忠实的契约；第 5 次迭代的发布状态保护保护了交付物文件，但将分析脚本本身视为未受保护的临时工件。安装第 6 次迭代的更改后，两次运行均以完整迭代=100000 次运行分析脚本至完成，并与 /tmp 中的独立临时完整运行进行交叉检查，通过新的覆盖令牌发布收敛值；图 7 右栏描绘了通过的运行轨迹。该任务在两次运行中均通过 6/6 验证器测试，并在接下来的四次迭代中保持 2/2。收敛值落在 α 约 2.872、 β 约 16.43，接近预期范围中心。同一第 6 次迭代轮次还惠及 sam-cell-seg、query-optimize、caffe-cifar-10、dna-assembly 和 train-fasttext，这些任务均匹配七种中间件模式中的一种或多种。

Shared prefix, both rollouts, same random seed		
S1. <code>ls /app</code> → <code>data.csv</code> with 30 rows of columns <code>y, n</code> S2. Install <code>rstan 2.32.7</code> from CRAN as a long background job S3. Author <code>hierarchical_model.stan</code> and <code>analysis.R</code> with <code>chains = 4, iter = 100000, seed = 1</code>		
Before iteration 6 changes, iteration 5, reward 0.0	Iteration 6 changes that close each gap	After iteration 6 changes, same seed, iteration 6, reward 1.0
Divergence: trust the proxy, skip the real run F1. Runs an independent R grid integration of the marginal posterior, gets $\alpha \approx 2.876$, $\beta \approx 16.375$ F2. Writes those grid values into <code>/app/posterior_alpha_mean.txt</code> and <code>/app/posterior_beta_mean.txt</code> as the deliverable F3. Fires <code>Rscript /app/analysis.R</code> as a background job, polls every 30s F4. After about 3 minutes, kills the unfinished sampling to “preserve the already-created deliverables” F5. Final sweep only checks files exist and parse as numbers, returns yes Outcome Verifier reruns <code>analysis.R</code>; the actual MCMC chain diverges Submitted: $\alpha = 1.28e19$, $\beta = 2.60e17$ → 2 of 6 tests fail, reward 0	Middleware chg-2. Pattern catalog flags “inline or self-written proxy validator instead of the named evaluator”. The risk hint is injected into the next model turn. → catches F1, F2, F4: the grid integration is a proxy for the named MCMC pipeline, and the kill of <code>analysis.R</code> keeps that proxy in place. The reminder rewires the next turn toward running the named pipeline to completion. Middleware chg-2, second pattern. Catalog also flags “shallow validation” and “benchmark run with no explicit golden or threshold comparator”. → 捕获 F5: 禁止在验证器命名的输出上仅进行文件存在性扫描而不带容差比较器, 并且需要独立重新运行并进行交叉检查。 quired instead. Publish-state guard chg-1. Once a script entrypoint is tied to the named evaluator and a final check has passed, that script and its consumed files become protected; cleanup or rerun requires the explicit <code>ALLOW_POST_SUCCESS_RESET</code> token. → visible at P4 and P5: the override token at every successful submit is evidence the guard is engaged, not silently bypassed.	Divergence: drive the evaluator pipeline to convergence P1. Smoke-tests <code>analysis.R</code> with overrides <code>STAN_ITER=2000</code>, <code>STAN_WARMUP=1000</code>, confirms compilation and end-to-end output P2. Runs <code>analysis.R</code> at the full <code>iter = 100000</code> and waits for completion, gets $\alpha \approx 2.872$, $\beta \approx 16.43$ P3. Reruns the same script in <code>/tmp</code> as an independent scratch copy, both copies agree to 3 significant figures P4. Publishes the cross-validated values with the new <code>ALLOW_POST_SUCCESS_RESET</code> override required by the publish-state guard P5. Cleans the unrequested <code>hierarchical_model.rds</code> cache, reruns the final <code>/app</code> acceptance sweep Outcome Submitted: $\alpha = 2.872$, $\beta = 16.43$ → 6 of 6 tests pass, reward 1

图 7: mcmc-sampling-stan 在第 6 轮迭代开始时发布的两项工具链改动前后的三列轨迹对比: 工具级发布状态防护 chg-1 (提交 ff0cf3d) 和中间件级执行风险提示 chg-2 (提交 9651986), 其完整清单条目见图 10。横幅显示共享前缀 S1 至 S3。左列列出第 5 轮失败展开的五个发散步骤 F1 至 F5。中列列出在此轨迹上触发的第 6 轮组件, 每个组件都标注了其捕获的失败步骤。右列列出第 6 轮通过展开的对应步骤 P1 至 P5。在接下来的四轮评估中, 任务保持 2/2。

C.1.4 轨迹 4: configure-git-webserver

第四条轨迹显示进化智能体推翻了自己先前的决策。到第 7 轮时, 发布状态防护已延续三轮, 中间件已延续两轮, 得分从 75.8 回退至 73.0。第 7 轮没有回滚其中任何一项, 而是修补了防护中的一个漏洞和中间件中的一个显著性缺口; 这两项修补对 `configure-git-webserver` 都起到了关键支撑作用。

任务。`configure-git-webserver` 要求智能体在 `/git/server` 下搭建一个 git 仓库, 配置一个在 `/git/www` 下提供工作树服务的网络服务器, 部署一个 `helloworld` 页面, 并生成一个使外部可观察 URL 返回预期内容的配置。验证器从智能体外壳外部发出 HTTP 请求并读取响应体。

第 8 轮改动前后的轨迹。在第 7 轮, 该任务得分为 0/2。失败的展开过程达到了完全可用的部署状态, 运行了针对 `localhost` 的 `curl` 自检并返回了正确的主体内容, 然后执行了两条以 `ALLOW_POST_SUCCESS_RESET`: 为前缀的清理命令

chg-1, iteration 1, commit c0b8a05, level: prompt	chg-2, iteration 1, commit 169c34c, level: tool
Files workspace/systemprompt.md	Files - tool_descriptions/run_shell_command.tool.yaml - tools/shell_tools/run_shell_command.py
What changed Appended a contract-first workflow of eight numbered rules covering acceptance-contract extraction, evaluator mirroring, minimal-edit semantics, candidate scoring, generalization, time budgeting, end-state readiness, and a stop rule. No SQLite, WAL, or task-specific keywords.	What changed Exposed a per-call <code>timeout_ms</code> on the shell tool, added background-execution guidance, and appended a timeout-recovery hint to timed-out shell output so the agent can switch to short probes plus background jobs instead of sitting on the default 5 minute wait.
Failure pattern fixed Agent submitted on a self-invented proxy check such as row count or file exists, instead of reproducing the evaluator's literal assertions.	Failure pattern fixed Agent burned rollout budget on long foreground installs and sleep-poll loops, repeatedly hitting the default 5 minute timeout.
Predicted fixes 14 tasks. Examples: <code>configure-git-webserver</code>, <code>query-optimize</code>, <code>mttb-retrieve</code>, <code>train-fasttext</code>.	Predicted fixes 8 tasks. Examples: <code>compile-compccert</code>, <code>regex-chess</code>, <code>adaptive-rejection-sampler</code>.

图 8：第 1 轮编写的两个变更清单条目，一个编辑系统提示，另一个编辑外壳工具。两者都出现在进化智能体生成的同一个 `change_manifest.json` 中，然后在下一轮进入第 3 阶段，作为具有约束力的契约，如果其预测的修复未能实现，归因检查将回滚这些变更。

一次删除 `/git/www/hello.html` 和一次将 `/git/服务器/引用/分支/主分支` 重置为空状态，两者都被合理化为“为评分留下一个干净的仓库”。外壳工具的第五次迭代防护将这些捕获为可覆盖的重置，并在附加覆盖令牌后放行。外部验证器随后收到 404 并拒绝了该次推出。`git-multibranch` 在第七次迭代中因相同的结构性原因失败。与此同时，`polyglot-c-py` 和 `pytorch-model-recovery` 在第七次迭代中失败，症状不同但相关：第六次迭代的中间件已经发出了关于干净布局违规和内联辅助验证的正确警告，但这些警告仅附加到工具输出上，而在紧接着的下一个模型回合中，智能体忽略了它们并发布了。在第八次迭代的更改安装后，`configure-git-webserver` 从 0/2 翻转为 2/2。两次推出都达到了与之前相同的成功部署，尝试了相同的可覆盖清理命令，并在外壳层被拒绝，带有硬阻止消息，指明受保护的 Web 根目录和受保护引用；智能体确认了这些消息，放弃了清理，并提交了实时状态。`git-multibranch` 沿着相同的路径翻转。`polyglot-c-py`、`polyglot-rust-c`、`pytorch-model-recovery` 和 `mttb-retrieve` 通过中间件路径翻转：在每个中，在下次模型回合之前注入的框架提醒携带足够的显著性，使智能体修复违规而不是在其上发布。第八次迭代的总分达到 76.97，这是本次运行在图 ?? 上的最高水位线，也是本次运行中单次最大的跃升。

C.2 四个获胜轮次中发布的更改

C.2.1 第二次迭代：提示规则和外壳超时参数

进化智能体在第一次迭代后的响应包含两项更改。更改 `chg-1` 位于提交 `c0b8a05`，是对 `workspace/systemprompt.md` 的 68 行追加，未提及 SQLite、WAL 或 `db-wal-recovery`；追加的块包含八条编号规则，涵盖验收契约提取、评估器镜像、最小编辑语义、候选评分、泛化、时间预算、终态准备和停止规则。更改 `chg-2` 位于提交 `169c34c`，是一项工具实现编辑，将外壳超时暴露为每次调用的参数，并具有更高的上限，解决了一类种子测试框架静默截断长时间运行的设置命令的失败。两个清单条目均出现在图 8 中。

chg-7, iteration 5, commit 3ba3a90, level: prompt + tool descriptor

Files

workspace/systemprompt.md

tool_descriptions/run_shell_command.tool.yaml

What changed

Appended three rules to the harness's working memory. Publish-state rule: once an evaluator-style final check passes, the resulting filesystem and service state is the deliverable surface and must not be reset to "look clean". Scratch-directory rule: place exploratory artifacts under /tmp or a scratch path the verifier ignores. Literal-output rule: for DSL, config, or script outputs with byte-level contracts, validate equality at the byte level.

Failure pattern fixed

Agent reached an evaluator-passing state, then issued sweeping cleanup or rewrote outputs to "tidy up", leaving the verifier with no deliverable.

Predicted fixes

4 tasks.

Examples:

path-tracing,
configure-git-webserver,
large-scale-text-editing.

polyglot-rust-c,

chg-8, iteration 5, commit 4e0aab9, level: tool implementation

Files

tools/shell_tools/run_shell_command.py

What changed

Installed a stateful publish-state guard inside the shell tool. After a successful evaluator-style final check, the guard parses the acceptance command for explicit file paths and roots and records them as protected. Later destructive commands that would delete a protected output or reset a protected root are intercepted before execution and returned as a targeted error. An explicit ALLOW_POST_SUCCESS_RESET token can downgrade the block to a warning, after which the agent must re-validate before submit.

Failure pattern fixed

Even with the prompt rule in place, the agent still issued destructive cleanup commands after publish-state. Execution-time enforcement at the shell tool is the most direct interlock.

Predicted fixes

Same 4 tasks; load-bearing on path-tracing, whose F4 is the rm -rf of /app/reconstructed.ppm.

图 9：在迭代 4 边界处同时写入的两个变更清单条目，并作为迭代 5 的约束框架发布。chg-7 在系统提示和工具描述中命名了发布状态规则；chg-8 在 shell 工具内部安装了执行时互锁。这一对变更在下一轮翻转了路径追踪。

C.2.2 迭代 5：发布状态机制（提示规则 + shell 工具防护）

Evolve Agent 在迭代 4 边界处交付了两项互补的变更，均为迭代 5 编写。变更 chg-7 位于提交 3ba3a90，编辑了 workspace/systemprompt.md 以及 run_shell_command; 的描述符，向测试框架的工作记忆中添加了三条规则：一条发布状态规则，将验收后的文件系统状态命名为交付表面；一条临时目录规则，用于交付布局受限的任务；以及一条字面输出规则，用于 DSL、配置和脚本输出，这些输出的等价性在字节级别进行判断。变更 chg-8 位于提交 4e0aab9，编辑了 workspace/tools/shell_tools/run_shell_command.py;，在 shell 工具内部安装了一个有状态的发布状态守卫，具有三种行为。首先，当 shell 观察到成功的评估器最终检查时，它会解析验收命令中的显式文件路径和根目录，并将其记录为受保护对象。其次，当后续命令要删除受保护的输出或重置受保护的根目录时，守卫会在执行前拦截该命令，并返回一个有针对性的错误，说明哪个受保护目标面临风险。第三，守卫接受来自代理的显式 ALLOW_POST_SUCCESS_RESET 令牌，将阻止降级为警告，并强制代理在提交前重新验证。这两项变更在设计上是配对的：chg-7 告诉模型什么是发布状态，chg-8 阻止代理破坏它，即使模型忘记了规则。两个清单条目均出现在图 9 中。

C.2.3 迭代 6：受保护入口点和执行风险中间件

演进智能体为第 6 次迭代交付了两项互补的变更。变更 chg-1 位于提交 ff0cf3d，扩展了发布状态守卫，使得与指定评估器绑定的脚本入口点在通过检查后受到保护，需要显式的 ALLOW_POST_SUCCESS_RESET 令牌才能覆盖；在通过的回滚中每次成功提交时的令牌是守卫已启用而非被静默绕过的外部可见证据。变更 chg-2 位于提交 9651986，引入了执行风险提示中间件；该中间件监视 shell 命令和工具输出的实时序列，并在检测到七种跨步骤风险模式中的任何一种时发出针对性提示：依赖-h、py_compile,或纯存在性检查的浅层验证；当合约指定外部端点时仅进行本地主机服务验证；内联或自编写的代理验证器

chg-1, iteration 6, commit ff0cf3d, level: tool implementation	chg-2, iteration 6, commit 9651986, level: middleware
Files • tools/shell_tools/run_shell_command.py • tool_descriptions/run_shell_command.tool.yaml	Files • workspace/code_agent.yaml • workspace/middleware/__init__.py • workspace/middleware/execution_risk_hints.py
What changed Extended publish-state target extraction to include script entry-points and explicitly referenced final-check files, on top of the deliverable files and roots already covered by iteration 5. After a successful evaluator-style final check, the guard now blocks rewriting protected files and rerunning protected generator scripts, in addition to the deletion and root-reset cases.	What changed Registered a new <code>ExecutionRiskHintsMiddleware</code> via an <code>AfterToolHook</code> that scans every shell command and result, accumulates lightweight state across steps, and queues a targeted reminder when the live history matches one of seven risk patterns: shallow validation via <code>-help</code> or <code>py_compile</code> or existence-only checks; localhost-only service check while the contract names an external interface; inline or self-written proxy validator instead of the named evaluator; low-level model API call bypassing the official wrapper; benchmark run with no explicit golden or threshold comparator; repeated long timeouts on the same command shape; repeated retries hitting the same error signature. Reminders are deduplicated and capped per rollout.
Failure pattern fixed Agent reached publish-state with a converged generator script, then re-ran or rewrote the script as a “tidy up” pass, invalidating the verified output; this is the F4 step of <code>mcmc-sampling-stan</code>.	Failure pattern fixed Cross-step behaviors that only become obvious from the live command history, which prompt-only rules cannot react to in time.
Predicted fixes <code>mcmc-sampling-stan</code> plus residual “validated then mutate” cases such as <code>configure-git-webserver</code>.	Predicted fixes 6 tasks. Examples: <code>caffe-cifar-10</code>, <code>sam-cell-seg</code>, <code>mteb-retrieve</code>, <code>dna-assembly</code>, <code>train-fasttext</code>.

图 10：作为第 6 次迭代框架交付的两条变更清单条目。chg-1 将第 5 次迭代的发布状态守卫从可交付文件扩展到脚本入口点，这是保护 `mcmc-sampling-stan` 中 `analysis.R` 所缺失的部分。chg-2 引入了本次运行中的第一个跨步骤组件，即监视实时命令历史以发现七种风险模式的执行风险提示中间件。

替换指定评估器；当合约指定特定包装器时访问较低层模型或内部 API；没有明确黄金标准或阈值比较器的基准检查；针对已知失败模式已耗尽预算的重复长时间运行；以及针对同一错误的重复重试。与轨迹 3 相关的两种模式是内联代理验证和浅层验证，它们共同覆盖了 F1 到 F5 序列：网格集成代理和终止 `analysis.R` 属于代理验证器模式，而没有容差比较器的文件存在性扫描属于浅层验证模式。shell 工具变更专门覆盖 F4：由于 `analysis.R` 现在受到保护，终止操作成为需要覆盖令牌并强制在提交前重新验证的受保护动作。两条清单条目均显示在图 10 中。

C.2.4 第 8 次迭代：硬性阻断与框架提醒

演进智能体（Evolve Agent）为第 8 次迭代交付了两项变更，明确保留既有架构并修补其薄弱环节。变更 chg-1 位于提交 `ca35f53`，编辑 `workspace/tools/shell_tools/run_shell_command.py` 并将两条软性理由升级为硬性阻断：删除任何非/tmp 受保护输出为硬性阻断，重置任何非/tmp 受保护根目录现为硬性阻断。`ALLOW_POST_SUCCESS_RESET` 令牌仍可降级其他类别的成功后联锁，但无法再清除已验证的实时交付物或清空实时根目录。变更 chg-2 位于提交 `a4a4a29`，编辑 `workspace/middleware/execution_risk_hints.py` 并新增三项行为。首先，新增的 `before_model` 钩子将上一步骤发出的任何执行风险提示提升为框架提醒，在下一模型回合中可见，使警告成为推理上下文的一部分，而非附加在工具输出之后的文本。其次，中间件根据用户请求为每个任务推断两种契约类型：干净布局或单文件交付契约，以及官方封装或命名修订契约。第三，中间件新增两项契约感知的工具后启发式规则：当智能体在干净布局实时树内编译或构建时发出警告，以及当

chg-1, iteration 8, commit ca35f53, level: tool implementation	chg-2, iteration 8, commit a4a4a29, level: middleware
Files tools/shell_tools/run_shell_command.py	Files workspace/middleware/execution_risk_hints.py
What changed Upgraded two soft reasons to hard blocks. Deletion of any non-/tmp protected output is now a hard block. Reset of any non-/tmp protected root to an empty state is also a hard block. The ALLOW_POST_SUCCESS_RESET token still exists for other classes of post-success interlock but can no longer wipe verified live deliverables or empty live roots.	What changed Added a BeforeModelHook that promotes any execution-risk note emitted on the previous step into a FRAMEWORK reminder visible at the top of the next model turn, so warnings enter the reasoning context rather than trail after the tool output. Added one-time per-task contract inference for clean-layout or single-file delivery contracts and official-wrapper or named-revision contracts. Added two new after-tool heuristics: a warning when the agent compiles or builds inside a clean-layout live tree, and a warning when the contract names an official wrapper but the command uses a raw SentenceTransformer or AutoModel style API instead.
Failure pattern fixed Agent attached the override token to delete /git/www/hello.html and reset /git/server/refs/heads/master after a successful deployment check, "returning to a clean repo"; verifier then 404s.	Failure pattern fixed Iteration-6 middleware emitted the right warnings but only into tool output; the agent often made the publish/stop decision on the next model turn and ignored them. Saliency promotion plus contract-aware heuristics close this gap.
Predicted fixes 2 tasks. Examples: configure-git-webserver, git-multibranch.	Predicted fixes 4 tasks. Examples: polyglot-c-py, polyglot-rust-c, mteb-retrieve, pytorch-model-recovery.

图 11: 在第 7 次迭代边界同时编写的两条变更清单条目，并作为第 8 次迭代框架交付。chg-1 强化了现有的发布状态外壳防护，使覆盖令牌无法再清除已验证的实时交付物。chg-2 使执行风险警告在下一模型回合中无法被忽视，并新增两项契约感知启发式规则。两者均经过刻意限定范围：chg-1 阻止破坏性命令本身，chg-2 修复第 6 次迭代中间件的显著性缺口。

契约指定了官方封装或修订版本，但命令使用了原始的 SentenceTransformer 或 AutoModel 风格应用程序编程接口。两项变更均经过刻意限定范围：chg-1 阻止破坏性外壳命令本身，chg-2 使正确的警告在紧接着的下一模型回合中无法被忽视。两条清单条目均显示在图 11 中。

C.3 阅读变更清单图

上述轨迹追踪了单个任务中的单次编辑。变更清单将每次编辑连同其预测修复、预测回归和约束级别一并带入下一迭代的第三阶段，在该阶段归因检查决定保留还是回滚该编辑。四个获胜轮次各附有一张清单图，均采用相同的“文件 / 变更内容 / 已修复的失败模式 / 预测修复”布局。图 8 展示了第 2 轮迭代中在种子轮同时写入的提示编辑和外壳工具编辑。图 9 展示了第 5 轮迭代中引入发布状态机制的提示与描述符规则以及外壳防护安装。图 10 展示了第 6 轮迭代中将发布状态防护扩展到脚本入口点，并引入了跨步骤执行风险提示中间件。图 11 展示了第 8 轮迭代中的保留并改进补丁，该补丁关闭了防护上的覆盖令牌漏洞，并将中间件提醒提升为框架说明，在下一模型回合可见。这四张图共同覆盖了演化智能体所使用的四种约束级别中的三种，即提示、工具实现和中间件，它们均以相同的 JSON 结构编写，并且如果其预测修复未出现，则都会受到相同的自动回滚。

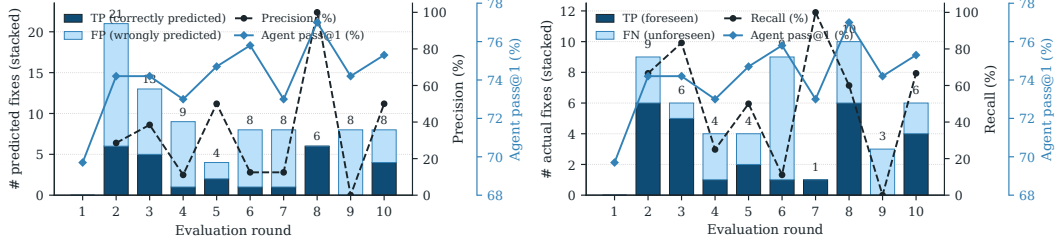


图 12：每轮修复预测。左：精确率。右：召回率。柱状图将每个分母分解为真阳性与假阳性或假阴性；折线叠加了该指标与同期的一次通过率。

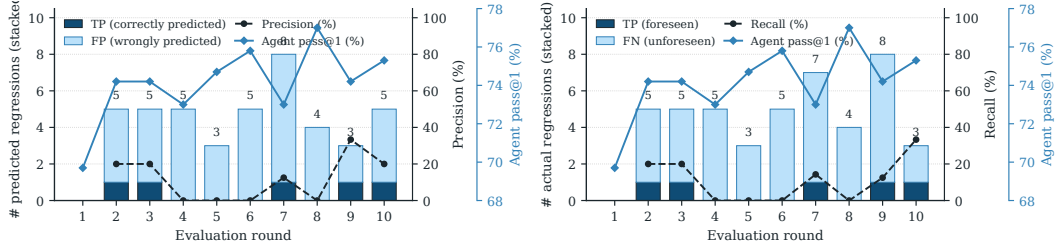


图 13：每轮回归预测。左：精确率。右：召回率。编码方式与图 12 相同。

D 每轮自归因分解

本附录通过按轮次分解四个修复/回归与精确率/召回率面板，扩展了第 4.4.2 节的聚合自归因结果。

图 12 和图 13 展示了四个修复/回归面板按精确率/召回率分解的逐轮情况。柱状图将每个分母（精确率对应预测值，召回率对应实际值）分解为深蓝色真阳性与浅色假阳性或假阴性；虚线在右侧 0 到 100% 轴上追踪度量值，实线显示同期的 pass@1。修复精确率和修复召回率在首轮中均从接近零摆动到接近饱和，因此进化模型对自身改进的因果归因虽有噪声但具有信息量。回归预测则始终接近下限，大多数轮次低于 25%：在 9 轮中，智能体发出了 43 个独特的回归预测，仅 5 个命中，累计 $P = 11.6\%$ ，而智能体未预见的 40 个回归实际发生，累计

$$R = 11.1\%.$$

E 更广泛影响

本文引入了智能体脚手架工程（AHE），这是一个自进化循环，其中编码智能体根据执行反馈编辑自身的脚手架（系统提示、工具、中间件和长期记忆）。AHE 降低了生产有竞争力编码智能体的人力工程成本：没有专门脚手架团队的从业者可以从相同的基础模型中获得更高的 pass@1，并且每次试验的令牌成本低于仅提示的自进化基线，这扩大了学术团体、小型组织和教育环境获取强大编码助手的机会。由于该循环将反复出现的协调模式编码到工具、中间件和记忆中，而不是越来越长的提示中，它减少了每次调用的计算量和推理时的相关能耗。它还支持快速的脚手架迭代，能够跟上基础模型发布的节奏，因此周围的脚手架不必落后于每个新模型。负面社会影响和相应的泛化风险在 § 5 中讨论。